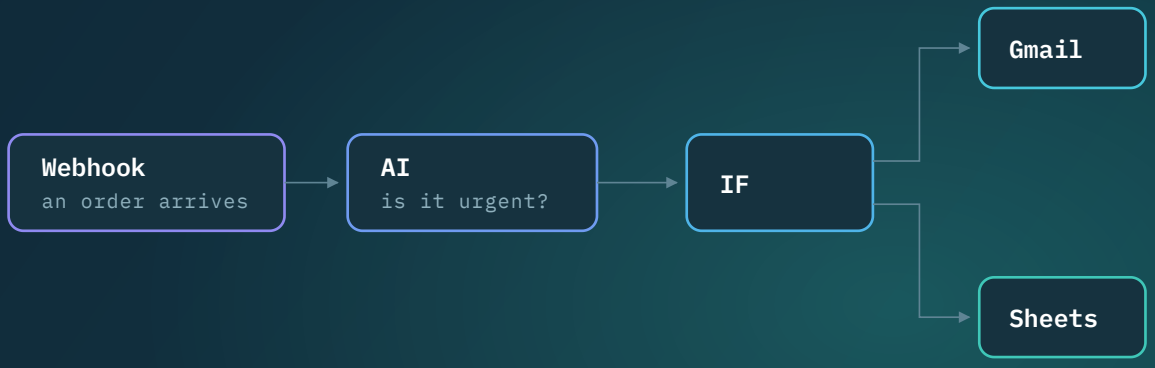
ELEC 3111 · GROUP 2 · SEMESTER PROJECT

A complete build guide

AI Automation Platform

Building n8n, from an empty folder

Every term explained, every part linked back to something you have already used, and every line of code you need — from a forty-line engine to a live demo.



1 · Make it run · 2 · Make it real · 3 · Usable · 4 · Self-starting · 5 · Do things · 6 · Ship it

Develop an AI automation platform similar to n8n

Schedule trigger · Webhook · HTTP · Google Docs, Sheets, Drive, Gmail · Code node · AI

Demonstration Week 14 · Report Week 15

8 people

6 phases · 15 stages · 22 diagrams

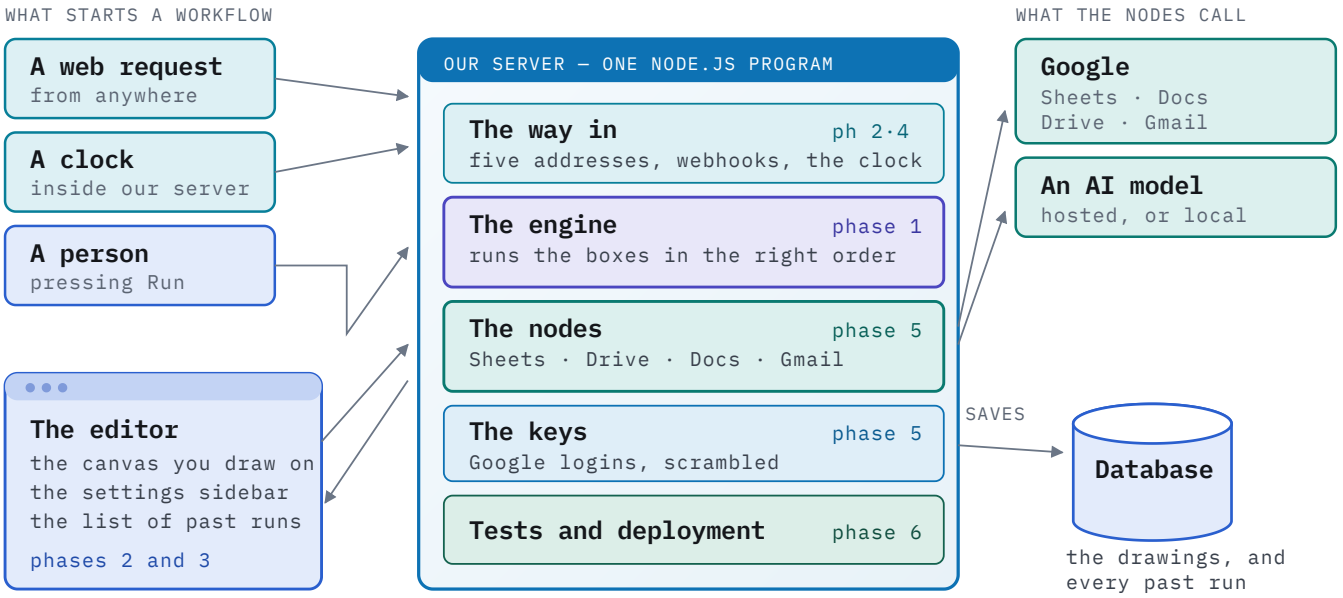
Contents

HOW TO READ THIS

Parts 0 and 1 explain how the thing works and are worth reading before anyone writes code. From Part 2 onwards each phase has **two pages**: the first says what we are doing and why, the second gives the actual commands and code. Each phase carries its own colour through every diagram belonging to it, so you always know where you are.

1.1 Everything we are building, on one page

This is the whole project. Nine boxes, and every one of them is built in a phase you can point at. **Nothing else exists.** When you feel lost later, come back to this page and find the box you are working on.



Everything travels the same way: something starts a workflow, the engine walks the drawing box by box, each box does one job — often calling Google or an AI model — and the run is written down.

Figure 1. The whole system. Each box is coloured by the phase that builds it, and every later diagram in this book is a close-up of one of them.

EVERY PHASE, FIRST PAGE

What and why

What we are doing, what it was in n8n, **who builds it**, what you need to know to build it, the tools, how to proceed, and a "done when".

EVERY PHASE, SECOND PAGE

How, exactly

The commands to type, the files to create, the code to write, and — where a website is involved — **every click, in order**.

THROUGHOUT

Words, where you meet them

Every term is defined the first time it appears, in a box that says what it is, what it does *there*, and how it connects to the rest.

Phase 1 · the engine · Phase 2 · database, server, canvas · Phase 3 · sidebar, expressions · Phase 4 · webhooks, clock · Phase 5 · nodes, keys · Phase 6 · tests, deployment

1.2 What a workflow really is

A workflow feels like a picture. It is actually **a piece of text** — a list of boxes and a list of lines, written down in JSON. The picture is only how we look at it.

> **FAMILIAR FROM N8N**
> When you clicked "Download" on a workflow, n8n handed you a `.json` file. That file *was* the workflow — you could email it to a friend, they could import it, and the same picture appeared on their screen. Nothing else was needed. That is the single most important thing to understand before we start.

workflow

What it is. One complete automation — the thing you switch on and leave running.

What it does in this section. It is the JSON on the right of Figure 2: a list of boxes and a list of lines, nothing more.

How it connects. The database stores one row per workflow; the engine is handed exactly this object and nothing else.

JSON

What it is. A plain-text way of writing data down. Curly braces for a thing, square brackets for a list.

What it does in this section. It is the format the drawing is saved in, and the format every node passes its data along in.

How it connects. The canvas produces it, the database keeps it, the engine reads it — it is the common language of the whole project.

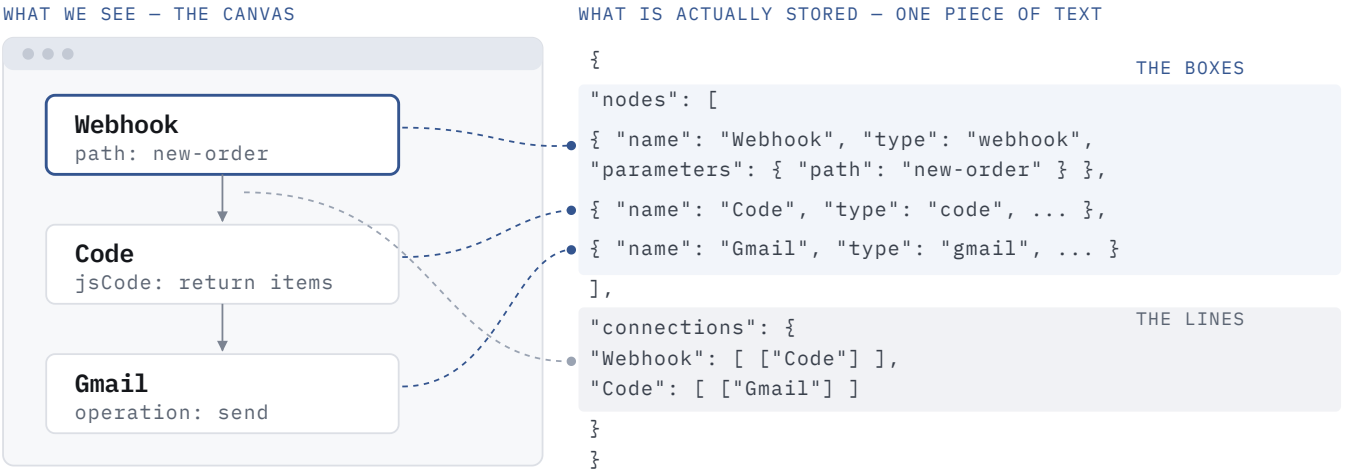


Figure 2. Every box you drag becomes one entry in **nodes**; every line you draw becomes one entry in **connections**. That is the whole file format — and it is all the engine ever receives.

WHY THIS IS GOOD NEWS

Because a workflow is just text, the two halves of our project can be built by different people at the same time. The canvas only has to *produce* this text; the engine only has to *read* it. Neither needs to know how the other works — they only have to agree on the shape above.

WHAT IT MEANS FOR US

Our database will store this text in a single column. Saving a workflow is one write; loading it is one read; making a copy is copying a row. Big commercial platforms do exactly the same thing, for exactly this reason.

1.3 What a node really is

A node is **one file that does two separate jobs**. Half of it *describes* itself, so the screen knows what settings to draw. The other half is an ordinary function that *does the work* when the engine calls it. Nothing else. Once this is clear, adding a node to our platform stops being mysterious.

node

What it is. One step of a workflow: a small object with a description and one function.

What it does in this section. The description tells the sidebar what settings to draw; the function does the actual job when the engine calls it.

How it connects. Everything in Phase 5 is a node. Because the shape never varies, the engine can run a node written five minutes ago.

parameters

What it is. The settings a node was given — the URL typed into a box, the operation chosen from a dropdown.

What it does in this section. They live on the node inside the workflow JSON, and the node reads them with `getNodeParameter`.

How it connects. Phase 3 makes the sidebar that fills them in; Phase 1 froze the way a node asks for them.

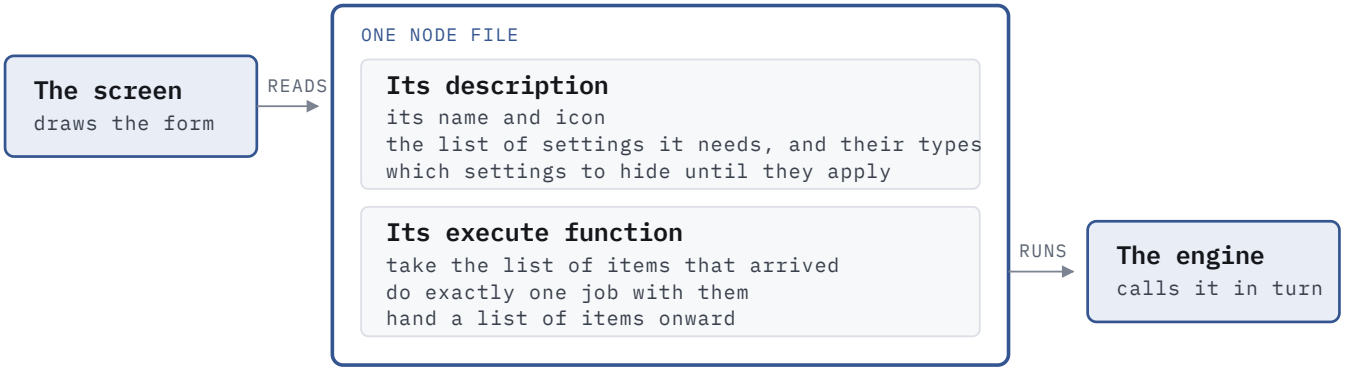


Figure 3. Two halves, two audiences. The screen never runs a node; the engine never draws anything. Neither half needs to know the other exists.

FAMILIAR FROM N8N

Click a Google Sheets node in n8n and the panel on the right appears instantly, with exactly the right fields. Change **Operation** from "Append" to "Read" and the fields change with it. n8n did not hand-build hundreds of panels — every node describes its own fields, and one piece of code turns any such description into a form. We are copying that idea deliberately, and it is the reason four of us can add nodes at the same time.

THE DESCRIPTION HALF

Data, not code

A plain list: "URL is a text box", "Method is a dropdown with four choices", "only show Body when Method is POST". The screen reads that list and draws the panel. Adding a field to a node means adding a line to this list — **never touching the editor**.

THE EXECUTE HALF

One job, done plainly

Items come in, the node does its single job — call an API, reshape some fields, split the flow in two — and items go out. A node never talks to the database, never draws anything, and never decides what runs next. That is the engine's business.

THE RULE THAT KEEPS US OUT OF TROUBLE

Every node in our platform obeys the same shape, whether it sends email through Gmail or simply adds a field. Because the shape never varies, the engine can run a node it has never seen before, and the screen can draw a settings panel for a node written five minutes ago. We agree this shape in Phase 1 and then **freeze it** — changing it later would break everyone's work at once.

1.4 What an item really is

An item is **one thing** — one spreadsheet row, one email, one order. Nodes never pass a single item; they always pass a **list** of them, even when the list is one item long. This sounds like a small detail. It is the decision that shapes every line of code we write.

item

What it is. One single thing travelling between nodes: one row, one email, one order.

What it does in this section. Nodes always pass a *list* of items, even when the list holds one thing.

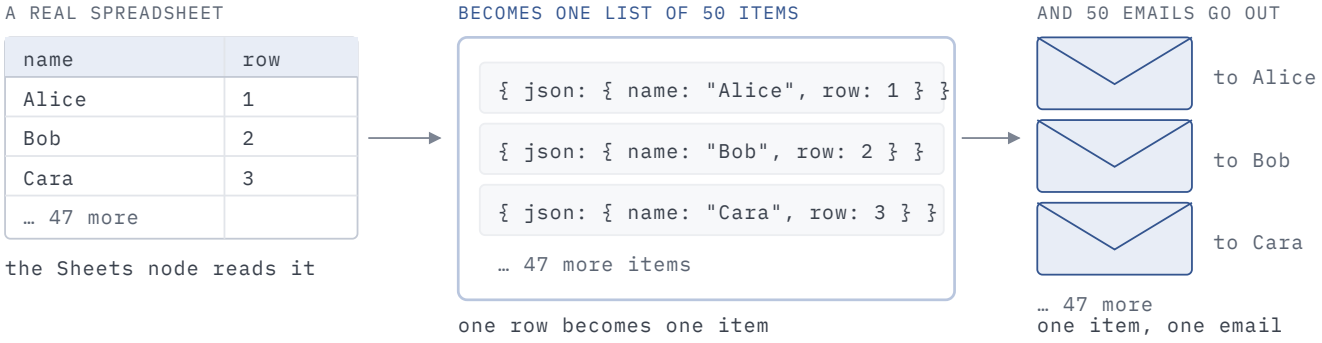
How it connects. This is why fifty spreadsheet rows become fifty emails with no extra code anywhere.

branch

What it is. One exit from a node. Most nodes have one; a node that splits the flow has two.

What it does in this section. A node returns one list per branch — the outer list is the exits, the inner list is the items.

How it connects. An empty branch stops everything after it, which is how the IF node makes half a workflow go quiet.



The Gmail node is called ONCE. It receives all fifty items and loops over them itself — we never write "repeat this node fifty times" anywhere, because the list already says so.

Figure 4. One read, fifty items, one send node, fifty emails — and no special code anywhere to make that happen.

FAMILIAR FROM N8N

Open any node's OUTPUT panel in n8n and the heading says something like *"5 items"*, with little numbered tabs you can click through. That is exactly this list. And it explains something you may have noticed: put a Gmail node after a node that produced five items, and n8n sends five emails without you configuring anything. The list is why.

INSIDE ONE ITEM

A labelled bag of values

Each item holds a `json` part — the ordinary fields, like a name and an email address — and optionally a `binary` part for files riding along, such as a PDF that a later node will attach to an email.

THE LIST OF LISTS

One list per exit

Most nodes have one exit, so they return one list. A node that splits the flow, like IF, has two exits and returns two lists — everything that passed the test, and everything that did not.

THE BUG EVERY ONE OF US WILL HIT AT LEAST ONCE

Returning one list where two were expected, or the other way round. Remember it as: **the outer list is the exits, the inner list is the items.** A normal node returns "one exit, containing all my items". When a node mysteriously sends nothing onward, or sends everything down the wrong branch, this is almost always why — and printing the data between nodes finds it in about a minute.

1.5 What the engine really is

An **engine** is a program that reads a set of instructions and carries them out in the right order. A chess engine reads a position; a search engine reads a query. **Our engine reads a workflow** — the JSON from section 1.2 — and calls each node's execute function at the right moment, with the right data. It is about forty lines long, and it is the heart of the whole project.

engine

What it is. A program that reads instructions and carries them out in the right order.

What it does in this section. Ours reads a workflow and calls each node's function at the right moment with the right data.

How it connects. Phase 1 builds it and then nobody touches it again — Phases 2, 4 and 6 only add new ways to *call* it.

trigger

What it is. A node with nothing before it: the thing that starts a workflow.

What it does in this section. The engine finds it, hands it one empty item, and the run begins from there.

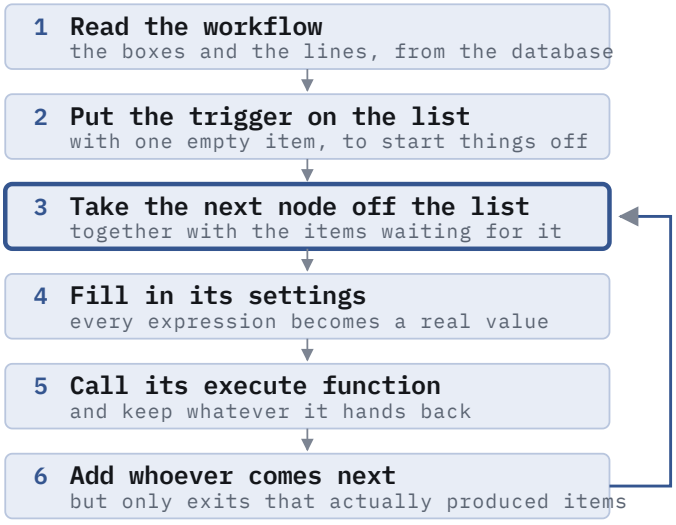
How it connects. Phase 4 builds two real ones — a web address and a clock — and neither requires changing the engine.

execution

What it is. One occasion on which a workflow actually ran, recorded so you can look back at it.

What it does in this section. The engine returns what every node produced; the server writes that down as a row.

How it connects. It is what the run list in the editor shows, and what the benchmark in Phase 6 measures.



THE SAME LOOP, RUNNING A REAL WORKFLOW

TICK	NODE RUN	PRODUCED	WAITING
1	Webhook	1 item	Code
2	Code	3 items	IF
3	IF	3 and 0	Gmail
4	Gmail	3 items	nothing

the list is empty, so the run is finished

Look at tick 3

The IF sent 3 items down its true exit and 0 down its false exit. Step 6 only adds exits that produced something — so the false side simply never ran.

Figure 5. Left: the loop. Right: the same loop actually running our demo workflow, one tick at a time, until the waiting list empties.

FAMILIAR FROM N8N

Press *Execute Workflow* in n8n and you can watch nodes light up one after another, each turning green as it finishes, branches that were not taken staying grey. You were watching this exact loop run. Our version will do the same thing — and because we wrote it, we will be able to explain every step of it in the report.

1.6 The canvas and the API — the two halves of the app

What looks like one application is really **two programs having a conversation**. The **canvas** runs inside your browser and only draws. The **server** runs somewhere else and does all the real work. They never share memory — they only send each other messages, and the list of messages they agree on is called the **API**.

> **FAMILIAR FROM N8N**
> Open `localhost:5678` in a second browser window and your workflows are all still there. Close the browser entirely, reopen it — still there. That is because the workflows were never *in* the page. The page asked the server for them, and the server read them from its database. Every screen you saw in n8n was assembled that way.

server

What it is. A program that waits and answers when something asks it a question. No screen — it listens on a numbered door called a port.

What it does in this section. Ours listens on port 5678 and holds the engine, the nodes and the keys.

How it connects. Everything except the canvas runs inside it, which is why Phase 2 builds it before anything else can be wired up.

API

What it is. The list of questions a server agrees to answer, and the exact wording each needs. A menu, not the kitchen.

What it does in this section. We publish one with five entries; Google publishes one our nodes will call.

How it connects. It is the only thing the canvas and the server have to agree on — freeze it in Phase 2 and both halves can change freely.

endpoint

What it is. One item on that menu: a verb plus a path, such as `POST /rest/workflows`.

What it does in this section. Each of our five does one job — list, fetch, create, save, run.

How it connects. Phase 4 adds two more of a different shape: `/webhook/:path` and its test twin.

database

What it is. A program whose only job is to remember things after everything else is switched off.

What it does in this section. Ours holds two tables: the drawings, and every past run.

How it connects. It is the reason a workflow is still there tomorrow, and the reason Phase 6 can deploy without losing anything.

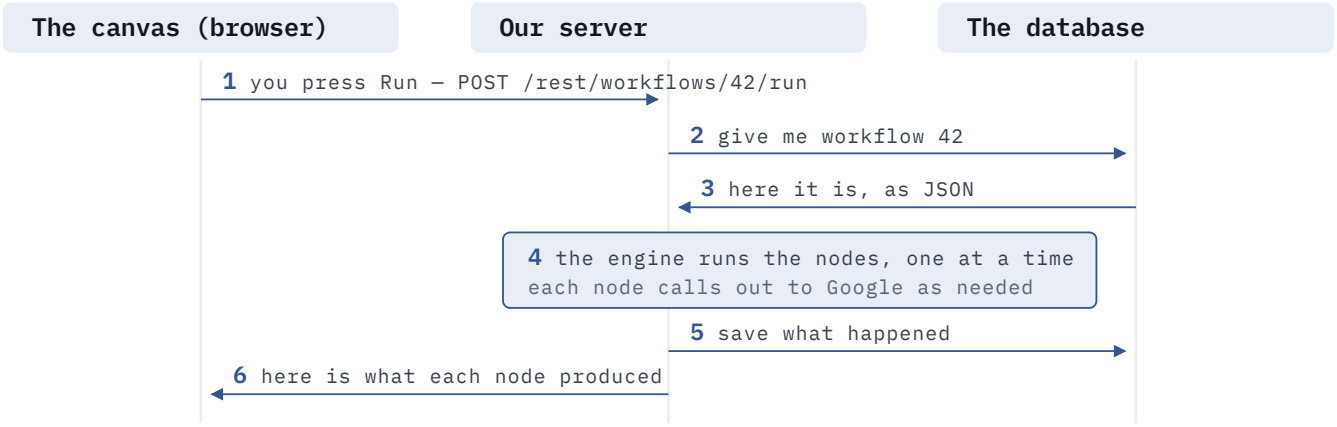


Figure 6. One click, six messages. The canvas never touches the database and never runs a node — it asks, and it displays the answer.

OUR OWN API

Five addresses is enough

List the workflows · fetch one · create one · save one · run one. That is the entire conversation our canvas needs for the first month. Each address has a verb (GET to fetch, POST to create, PATCH to change) and a path such as `/rest/workflows/42`.

OTHER PEOPLE'S APIS

The same idea, pointed outward

Google publishes an address that means "add this row to that spreadsheet". Our Sheets node sends a request to it and reads the answer. A node is, almost always, one carefully built request to somebody else's API.

WHY WE CARE ABOUT THIS SPLIT

Because it is what lets two people work at once without colliding. Whoever builds the canvas and whoever builds the server only have to agree on those five addresses — the shape of each message, and what comes back. After that, one can redesign the whole screen and the other can rewrite the whole engine, and neither breaks the other. We write that agreement down in Phase 2 and then leave it alone.

1.7 The six phases

Everything from here on is building. The work splits into six phases, and **each one ends in something we can run and show somebody**. From this page onward each phase carries its own colour, used in its heading and in every diagram that belongs to it, so you can always tell at a glance which part of the project you are looking at.

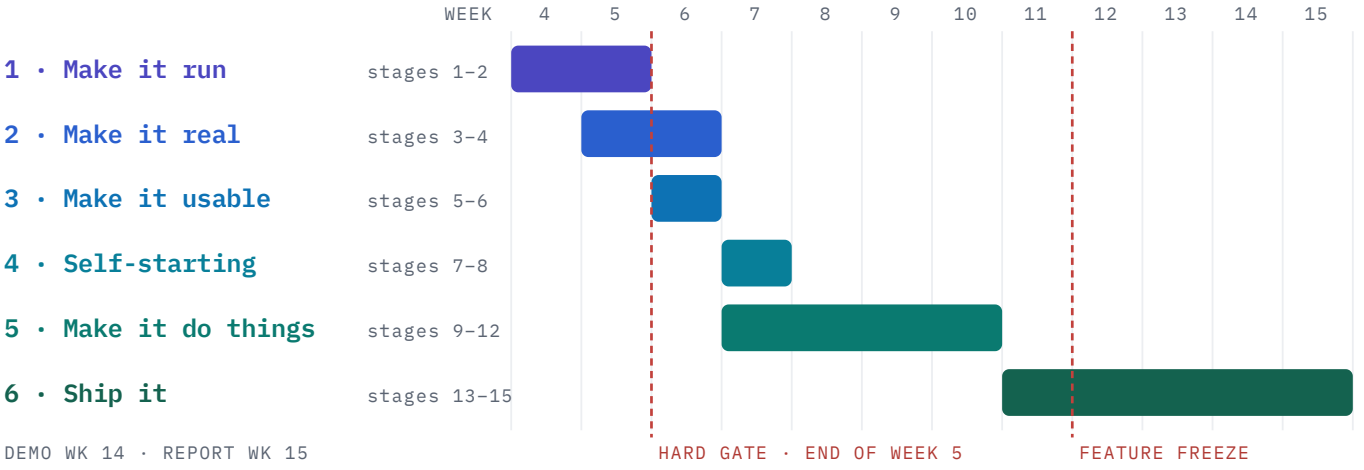


Figure 7. Two dates cannot move. By the end of Week 5 a workflow must run from end to end, however ugly it looks. At the end of Week 11 we stop adding features altogether.

1 · Make it run

A script that executes a workflow and prints the answer. No web page, no database.

2 · Make it real

Workflows are saved in a database and drawn on a canvas.

3 · Make it usable

Settings panels build themselves, and fields can carry live data.

4 · Self-starting

Workflows start on their own, from a web address or from a clock.

5 · Make it do things

Google permission, then the four Google nodes, the Code node and the AI node.

6 · Ship it

Online, measured against n8n, and written up.

THE RULE THAT PROTECTS THE SCHEDULE

If we fall behind we cut *nodes*, never phases. Losing the Drive node costs almost nothing. Losing Phase 3 means every later node needs a hand-built settings panel, and only one person can work at a time — which would cost us the whole of Weeks 8 to 10.

1.8 The toolkit — what we install, and when

The complete list for the semester. **Nobody needs to learn it all now** — each phase introduces two or three things, and about half of them are already built into Node.js. The guiding rule is on the right of the figure: build the thing the assignment is about, and borrow everything else.

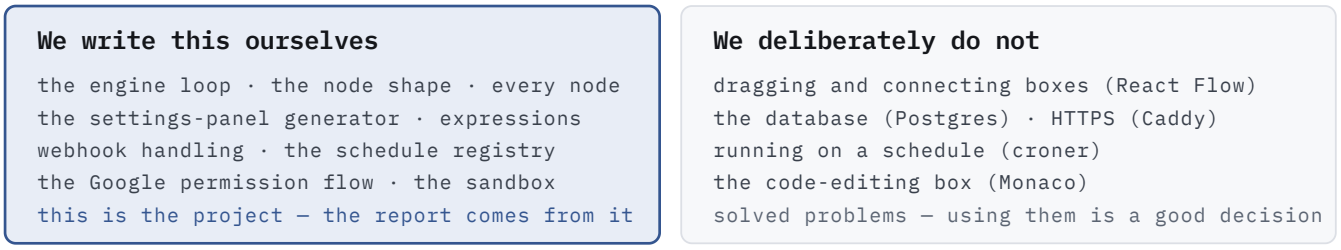


Figure 8. Borrowing the canvas library is not cheating; writing our own would cost a month and earn nothing. Borrowing the *engine* would be the mistake.

TOOL	WHAT IT IS FOR	PHASE
Node.js 22	The language everything is written in — the one we already learned this semester.	1
Git + GitHub	One shared folder with a full history of who changed what. Everyone works on a branch and merges.	1
TypeScript	JavaScript with labels on the data, so a mistake shows up in the editor instead of at three in the morning.	1
Docker Compose	Starts the database and the app together with one command, identically on all eight laptops.	2
PostgreSQL 16	The database. Remembers workflows and every past run.	2
Express	The server framework — it turns "when someone asks for this address, do that" into three lines.	2
React + Vite + React Flow	Builds the page in the browser, and gives us the canvas — dragging, drawing lines, panning and zooming, all solved.	2
Monaco	The code-editing box in the sidebar — literally the editor from VS Code.	3
croner	Runs a function on a schedule, with timezones handled properly.	4
Postman / curl + cloudflared	Send test requests to our own addresses, and give our laptop a temporary public address so outside services can reach it.	4
Google Cloud Console	The website where we switch on the four Google APIs and set up the permission screen. Free.	5
node:crypto, worker_threads	Both built into Node. One scrambles the stored Google keys; the other runs a user's JavaScript without risking the server.	5
Anthropic API	The AI node — one request, one response. Ollama can serve a local model instead, as in our weather project.	5
AWS EC2	One small rented computer that runs the finished platform, about $20 a month.	6
Caddy	Gets and renews the HTTPS certificate by itself. Three lines of configuration.	6
n8n (Docker)	The platform we compare ourselves against, set up the same way as ours so the comparison is fair.	6
k6	Sends measured traffic at both platforms so the comparison uses real numbers.	6

1.9 Install everything, click by click

WHO DOES THIS PART

all eight D2 checks everyone finished

WHAT YOU NEED TO KNOW

Nothing. This page assumes an empty laptop. Allow about an hour, once.

Do this before the first Monday. If any step fails, say so in the group chat the same day — half of these have a licence, an account or a restart in the way.

1 Node.js 22 — the language everything is written in

1. Go to `nodejs.org`
2. Click the big green `LTS` button — **not** Current
3. Run the installer, accept every default, and let it add Node to your PATH
4. Close every terminal window and open a new one — the PATH only updates in new windows

```
$ node -v     # must print v22.something
$ npm -v      # must print 10 or higher
```

2 VS Code, and the four extensions we all use

1. `code.visualstudio.com` → `Download` → install
2. Open it, then press `Ctrl+Shift+X` (`Cmd+Shift+X` on a Mac) for the Extensions panel
3. Search and install each: **ESLint · Prettier · Docker · GitLens**
4. `File → Preferences → Settings`, search `format on save`, tick it. Now nobody argues about spacing in a review.

3 Docker Desktop — how we run the database

1. `docker.com/products/docker-desktop` → download for your operating system
2. On Windows, when it asks, leave `Use WSL 2` ticked. It will make you restart.
3. Open Docker Desktop once and wait until the whale icon stops animating — nothing works until it has started
4. `Settings → Resources` → give it at least **4 GB** of memory

```
$ docker run --rm hello-world    # prints "Hello from Docker!"
```

4 Git and GitHub — one repository, eight people

1. Install Git from `git-scm.com` (on a Mac, `xcode-select --install` is enough)
2. Make a GitHub account each. **D2** then clicks `New repository` → name it `ai-automation` → `Private` → `Create`
3. `Settings → Collaborators → Add people` → add the other seven
4. `Settings → Branches → Add branch protection rule` → pattern `main` → tick `Require a pull request before merging` and `Require status checks to pass` → `Create`

```
$ git config --global user.name  "Your Name"
$ git config --global user.email "you@example.com"
$ git clone https://github.com/<org>/ai-automation && cd ai-automation
$ git checkout -b your-name/first-change   # never commit straight to main
```

5 Postman — for poking our own server, and Google's

1. `postman.com/downloads` → install → sign in (free)
2. `New` → `Collection` → name it `ai-automation`. Everything we try by hand lives here.
3. Inside it, `Add request` → set the verb to `POST`, the URL to `http://localhost:5678/rest/workflows`
4. Open the `Body` tab → choose `raw` → change `Text` to `JSON` → paste your JSON → `Send`
5. `Collection → … → Export` and commit the file, so all eight of us share the same requests

DONE WHEN

All five commands above print what they should, on all eight laptops. D2 keeps a tick-list — it is much cheaper to fix a broken install now than on the Thursday somebody is blocked by it.

PHASE 1 · MAKE IT RUN STAGES 1–2 · WEEKS 4–5 · LEAD: A1, REVIEWED BY A2

2.1 A workflow runs, from a single file

WHO BUILDS THIS PART

A1 A2 reviews

WHAT YOU NEED TO KNOW

Plain JavaScript — arrays, objects, **async/await**. No framework, no database, no React. If you can read a `for` loop and a promise, you can write this phase.

What we are doing. Writing the loop from section 1.5 as one small program, and agreeing the exact shape every node will have. **No web page, no database, no canvas** — just a file we run from the terminal and watch print real results.

engine.js

```
$ node engine.js
Start -> [ { } ]
Weather -> [ { "temp_C": "18",
"desc": "Partly cloudy" } ]
Shape -> [ { "city": "Newcastle",
"temp": 18 } ]
done
```

What just happened

A hard-coded workflow object, three node functions and the loop — all in one file, about forty lines.

It called a real weather API, passed the result to the next node, and printed what each one produced.

everything else is built around this

Figure 9. The whole product, on the first day. Every later phase adds a way to create, store, trigger or display this — never a second way to run it.

FAMILIAR FROM N8N

This is the moment n8n goes from magic to machinery. What you watched happen on the n8n canvas — nodes lighting up in order, data appearing in each output panel — is exactly what those printed lines are. The only difference is that n8n draws it prettily and we are printing it to a terminal.

TOOLS WE USE IN THIS PHASE

Node.js	the language we already learned — nothing new to install	**VS Code + Git**	one shared repository, everyone on a branch
TypeScript	used lightly, only to write down the node shape	**No frameworks**	deliberately — this phase is plain JavaScript in one file

HOW WE PROCEED

1. **One person writes the file.** A hard-coded workflow, two or three node functions, and the waiting-list loop. It should end by printing real data from a public weather API.
2. **All eight of us read it together** until nobody has a question. This is worth an hour of everyone's time — it is the only genuinely conceptual part of the whole project.
3. **We write the node shape down** in the repository: what a node receives, what it must hand back, and what it is never allowed to do (touch the database, draw anything, or decide what runs next).
4. **We build two boring nodes against it** — one that adds fields to each item, one that splits the flow in two. Nothing that can fail for reasons outside our control.
5. **We freeze the shape.** From here on, changing it needs everyone to agree, because a change breaks work already in progress.

DONE WHEN

Three nodes run in a chain, the splitting node sends data down the correct branch, and every one of us can explain the loop without notes.

WATCH OUT

Do not put eight people on this. A1 writes, A2 reads closely, and the other six work on the independent tracks in section 4.2. Parallel work on an unstable core produces eight rewrites — the most common way a large group fails quietly in week three.

2.1b Build it: the engine, line by line

Everything on this page goes into **one file**. Type it, run it, and you have a working workflow engine before lunch.

1 Create the workspace

```
$ mkdir ai-automation && cd ai-automation
$ npm init -y
$ npm pkg set type=module workspaces[0]="packages/*" workspaces[1]="apps/*"
$ mkdir -p packages/engine packages/nodes apps/api apps/editor
```

`type=module` lets us use `import`; `workspaces` is npm's own monorepo support — no extra tool to install.

2 The workflow, and two node types

packages/engine/engine.js

```
const workflow = {
  nodes: [
    { name: 'Start',   type: 'manualTrigger', parameters: {} },
    { name: 'Weather', type: 'httpRequest',
      parameters: { url: 'https://wttr.in/Newcastle?format=j1' } },
  ],
  // one entry per node: a list of output branches, each listing who it feeds
  connections: { 'Start': [['Weather']] },
};

const nodeTypes = {
  manualTrigger: {
    description: { name: 'manualTrigger', group: 'trigger', properties: [] },
    async execute() { return [[{ json: {} }]]; },
  },
  httpRequest: {
    description: { name: 'httpRequest', group: 'action', properties: [] },
    async execute(ctx) {
      const out = [];
      for (const item of ctx.getInputData()) {
        const res = await fetch(ctx.getNodeParameter('url'));
        out.push({ json: await res.json() });
      }
      return [out];              // one branch, containing every item
    },
  },
};
```

3 The loop from Figure 5 — the whole engine

packages/engine/engine.js — continued

```
export async function runWorkflow(workflow, nodeTypes) {
  const byName  = Object.fromEntries(workflow.nodes.map(n => [n.name, n]));
  const results = {};
  const trigger = workflow.nodes.find(n => nodeTypes[n.type].description.group === 'trigger');
  const ready   = [{ nodeName: trigger.name, items: [{ json: {} }] }];

  while (ready.length > 0) {
    const { nodeName, items } = ready.shift();          // 3 · take the next node
    const node = byName[nodeName];
    const ctx  = {                                       // 4 · what the node may ask for
      getInputData:     () => items,
      getNodeParameter: (name) => node.parameters[name],
    };

    const branches = await nodeTypes[node.type].execute(ctx);   // 5 · run it
    results[nodeName] = branches;
    console.log(nodeName, '->', JSON.stringify(branches[0]).slice(0, 80));

    const targets = workflow.connections[nodeName] || [];        // 6 · push what comes next
    branches.forEach((branchItems, i) => {
      if (branchItems.length === 0) return;              // an empty branch goes quiet
      for (const t of targets[i] || []) ready.push({ nodeName: t, items: branchItems });
    });
  }
  return results;
}

runWorkflow(workflow, nodeTypes).then(() => console.log('done'));
```

```
$ node packages/engine/engine.js
Start   -> [{}]
Weather -> [{"current_condition":[{"temp_C":"18"…
done
```

CHECK IT WORKS

Real weather appears. Then break it on purpose: make `httpRequest` return `[[]]` instead of `[out]` and confirm nothing after it runs. That is branch pruning, and you have just tested it.

PHASE 2 · MAKE IT REAL STAGES 3–4 · WEEKS 5–6 · LEAD: C1 · B1

2.2 Workflows survive a restart, and we can draw them

WHO BUILDS THIS PART

C1 B1 D2 helps with Docker

WHAT YOU NEED TO KNOW

SQL enough to create a table and write five queries · **HTTP verbs** (GET, POST, PATCH) · **React** basics — components, state, effects · running one Docker command.

What we are doing. Giving the engine a home. A **database** remembers workflows, a **server** hands them out through the five addresses from section 1.6, and a **canvas** lets us draw one instead of typing JSON by hand.

container

What it is. A program packed with everything it needs, so it behaves identically on every machine.

What it does in this section. We start Postgres as one, so nobody installs a database by hand.

How it connects. The same idea deploys the finished platform in Phase 6 — the container that runs on your laptop is the one that runs on the server.

migration

What it is. A file of SQL that creates or changes tables, kept in the repository and run in order.

What it does in this section. `schema.sql` is our first one; every later table change adds another.

How it connects. It means all eight laptops and the live server end up with exactly the same database shape.

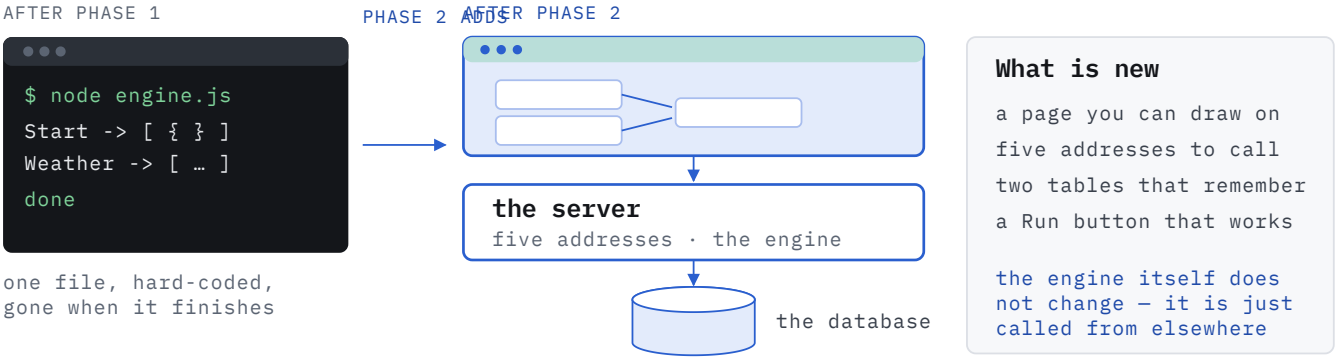


Figure 10. Phase 2 wraps phase 1 in the four things every web application has: a screen, a server, a database, and a way back. Nothing about the engine is rewritten.

FAMILIAR FROM N8N

Everything on this page is what n8n already gave you for free: a canvas that saved when you clicked away, a workflow list that survived a restart, and a Run button. You never thought about the database behind it — which is exactly the experience we are now building for ourselves.

TOOLS WE USE IN THIS PHASE

Docker Compose	starts the database with one command — nobody installs Postgres by hand	**PostgreSQL**	remembers the workflows and every past run
Express	the smallest useful server — readable end to end in an afternoon	**node-postgres**	talks to the database in plain SQL; no extra layer to learn
React + Vite	builds the page the canvas lives in	**React Flow**	dragging, connecting, panning and zooming — already solved

HOW WE PROCEED

1. **Start the database in a container.** One configuration file, one command, the same on every laptop.
2. **Two tables only:** one for workflows, one for runs. The whole drawing goes into a single column as the text from Figure 2 — which is what makes saving one write and loading one read.
3. **Build the five addresses:** list, fetch, create, save, run. That really is enough for the next month.
4. **Put React Flow on the page.** We write the boxes and the sidebar; we do not write dragging, zooming or line-drawing.

5. **Join the two halves** so the drawing saves to the database and the Run button executes it, showing what each node produced.

DONE WHEN

We can drag out two nodes, connect them, refresh the browser and find them still there — then press Run and see the result of each one.

WATCH OUT

Converting between React Flow's format and ours is the fiddly job here, and essentially every canvas bug for the rest of the semester will live in it. Write it once, carefully, with someone reading over your shoulder.

2.2b Build it: database, server and canvas

1 Start Postgres in a container

docker-compose.yml

```yaml
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: dev
      POSTGRES_DB: automation
    ports: ["5432:5432"]
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  pgdata: {}
```

```
$ docker compose up -d
$ psql postgresql://postgres:dev@localhost:5432/automation -f schema.sql
```

2 Two tables

schema.sql

```sql
CREATE TABLE workflows (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name        text NOT NULL,
  active      boolean NOT NULL DEFAULT false,
  nodes       jsonb NOT NULL DEFAULT '[]',      -- the boxes
  connections jsonb NOT NULL DEFAULT '{}',      -- the lines
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE executions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid REFERENCES workflows(id) ON DELETE CASCADE,
  status      text NOT NULL,                    -- running | success | error
  data        jsonb,                            -- what each node produced
  error       jsonb,
  started_at  timestamptz NOT NULL DEFAULT now(),
  finished_at timestamptz
);
```

3 The five routes

```
$ npm i express pg --workspace apps/api
```

apps/api/server.js

```
import express from 'express';
import pg from 'pg';
import { runWorkflow } from '../../packages/engine/engine.js';

const db  = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const app = express();
app.use(express.json());

app.get('/rest/workflows', async (req, res) =>
  res.json((await db.query('SELECT id, name, active FROM workflows ORDER BY name')).rows));

app.get('/rest/workflows/:id', async (req, res) =>
  res.json((await db.query('SELECT * FROM workflows WHERE id = $1', [req.params.id])).rows[0]));

app.post('/rest/workflows', async (req, res) => {
  const { name, nodes, connections } = req.body;
  const q = 'INSERT INTO workflows (name, nodes, connections) VALUES ($1, $2, $3) RETURNING *';
  res.json((await db.query(q, [name, nodes, connections])).rows[0]);
});

app.patch('/rest/workflows/:id', async (req, res) => {
  const { nodes, connections } = req.body;
  const q = 'UPDATE workflows SET nodes = $1, connections = $2 WHERE id = $3 RETURNING *';
  res.json((await db.query(q, [nodes, connections, req.params.id])).rows[0]);
});

app.post('/rest/workflows/:id/run', async (req, res) => {
  const wf  = (await db.query('SELECT * FROM workflows WHERE id = $1', [req.params.id])).rows[0];
  const run = await db.query("INSERT INTO executions (workflow_id, status) VALUES ($1,'running') RETURNING id",
[wf.id]);
  try {
    const data = await runWorkflow(wf, nodeTypes);
    await db.query("UPDATE executions SET status='success', data=$1, finished_at=now() WHERE id=$2", [data,
run.rows[0].id]);
    res.json(data);
  } catch (e) {
    await db.query("UPDATE executions SET status='error', error=$1, finished_at=now() WHERE id=$2", [{ message:
e.message }, run.rows[0].id]);
    res.status(500).json({ message: e.message });
  }
});

app.listen(5678, () => console.log('api on http://localhost:5678'));
```

CHECK IT WORKS
`curl -X POST localhost:5678/rest/workflows -H 'content-type: application/json' -d '{"name":"first","nodes":[],"connections":{}}'` — then restart the container and fetch it back. If it survives, Phase 2's hardest half is done.

2.2c Build it: the canvas, and the two functions that matter

4 Start the editor

```
$ npm create vite@latest apps/editor -- --template react-ts
$ cd apps/editor && npm i reactflow
$ npm run dev            # http://localhost:5173
```

5 Our format → React Flow's, and back

These two functions are where *every* canvas bug will live for the rest of the semester. Write them once, carefully, and give them a test.

apps/editor/src/convert.ts

```ts
import type { Node as RfNode, Edge } from 'reactflow';

export function toReactFlow(wf) {
  const nodes: RfNode[] = wf.nodes.map(n => ({
    id: n.name, type: 'workflowNode', position: n.position ?? { x: 0, y: 0 }, data: { node: n },
  }));
  const edges: Edge[] = Object.entries(wf.connections).flatMap(([from, branches]) =>
    (branches as string[][]).flatMap((targets, branchIndex) =>
      targets.map(to => ({
        id: `${from}:${branchIndex}->${to}`,
        source: from, target: to, sourceHandle: String(branchIndex),
      }))));
  return { nodes, edges };
}

export function fromReactFlow(nodes: RfNode[], edges: Edge[]) {
  const connections: Record&lt;string, string[][]&gt; = {};
  for (const e of edges) {
    const branch = Number(e.sourceHandle ?? 0);
    connections[e.source] ??= [];
    connections[e.source][branch] ??= [];
    connections[e.source][branch].push(e.target);
  }
  return {
    nodes: nodes.map(n => ({ ...n.data.node, position: n.position })),
    connections,
  };
}
```

6 Wire the canvas to the server

apps/editor/src/Editor.tsx — the parts that matter

```
const [nodes, setNodes, onNodesChange] = useNodesState([]);
const [edges, setEdges, onEdgesChange] = useEdgesState([]);

useEffect(() => {                                  // load
  fetch(`/rest/workflows/${id}`).then(r => r.json()).then(wf => {
    const { nodes, edges } = toReactFlow(wf);
    setNodes(nodes); setEdges(edges);
  });
}, [id]);

const save = () => fetch(`/rest/workflows/${id}`, {  // save
  method: 'PATCH',
  headers: { 'content-type': 'application/json' },
  body: JSON.stringify(fromReactFlow(nodes, edges)),
});

const run = () => fetch(`/rest/workflows/${id}/run`, { method: 'POST' })
  .then(r => r.json()).then(setRunResult);
```

Add `server: { proxy: { '/rest': 'http://localhost:5678' } }` to `vite.config.ts` so the browser can call the API without cross-origin trouble.

CHECK IT WORKS
Drag two nodes out, connect them, press Save, refresh the browser — they are still there. Press Run and the result of each node appears. That is Phase 2 finished and the Week 5 hard gate met.

PHASE 3 · MAKE IT USABLE STAGES 5–6 · WEEK 6 · LEAD: B2 · A1

2.3 Nodes describe themselves, and fields carry live data

WHO BUILDS THIS PART

B2 A1 B1 reviews

WHAT YOU NEED TO KNOW

React conditional rendering and controlled inputs · a little **regular-expression** reading for the expression resolver · above all, patience for detail: this is the fiddliest phase and the most valuable.

What we are doing. The two mechanisms that look small and decide everything: the settings panel that **draws itself**, and **expressions**.

expression

What it is. A small piece of text in a settings box, in double braces, replaced with real data just before the node runs.

What it does in this section. It is how one field becomes a different value for every item.

How it connects. The engine resolves it inside `getNodeParameter`, so nodes never see the braces — which is why Phase 1's contract did not need to change.

schema-driven UI

What it is. A screen built from a description of the data rather than hand-written for each case.

What it does in this section. Our sidebar reads a node's property list and draws the form from it.

How it connects. It is the single reason six people can add six nodes in Weeks 8–10 without touching the editor once.

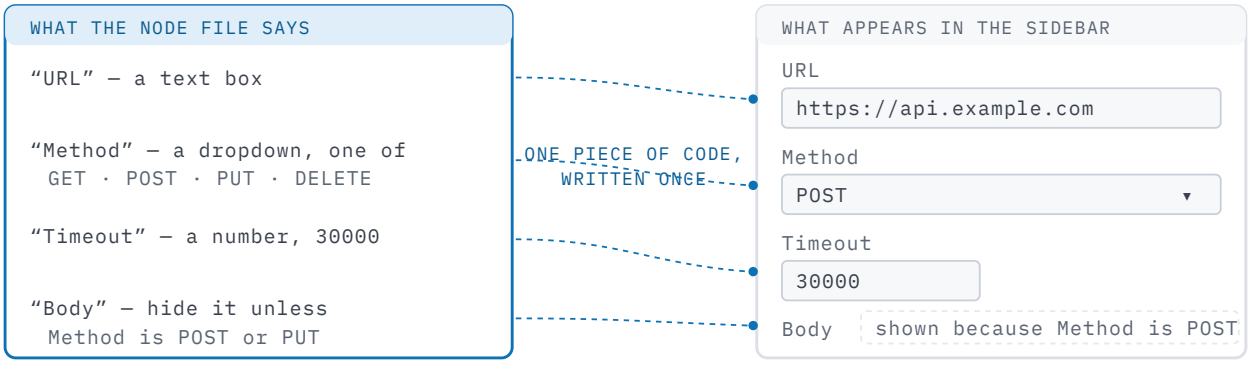


Figure 11. Four lines of description, four fields on screen — and the fourth appears only because Method happens to be POST. Nobody hand-builds a panel, ever.

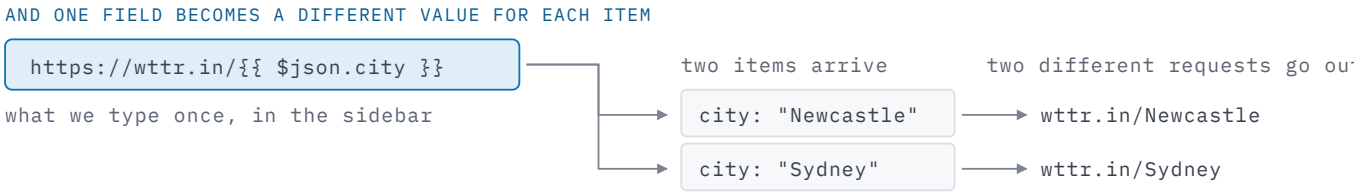


Figure 12. The engine fills the field in for every item separately, just before the node runs — the node never sees the braces.

FAMILIAR FROM N8N

Changing a node's **Operation** made half the fields disappear — that is the top figure. Dragging a field into a box inserted `{{ $json.something }}` — that is the bottom one.

TOOLS WE USE IN THIS PHASE

React	the one component that turns a description into a form	**Our own code**	no library does this for us — about a hundred lines, and the cleverest part of the project
Monaco	the editor from VS Code, dropped in later as the Code node's text box		

HOW WE PROCEED

1. **Agree the kinds of setting we support** — text, number, on/off, dropdown, code box — then write the single component that reads a description and produces those inputs.
2. **Add "only show this when…"**, so a panel changes as you pick an operation — this is what makes the Google nodes feel finished rather than overwhelming.
3. **Add expressions**, filled in for each item just before the node runs.

DONE WHEN
A brand-new node file appears in the sidebar with a full settings panel, and nobody touched the canvas code.

WATCH OUT
Skip this "to save time" and each of the next ten nodes needs a hand-built panel — turning Weeks 8–10 into a queue behind one person.

2.3b Build it: forms that draw themselves, and expressions

1 A node describes its own settings

packages/nodes/httpRequest.ts

```
export const httpRequest = {
  description: {
    name: 'httpRequest', displayName: 'HTTP Request', group: 'action',
    inputs: ['main'], outputs: ['main'],
    properties: [
      { displayName: 'Method', name: 'method', type: 'options', default: 'GET',
        options: [{ name: 'GET', value: 'GET' }, { name: 'POST', value: 'POST' }] },
      { displayName: 'URL', name: 'url', type: 'string', default: '' },
      { displayName: 'Timeout (ms)', name: 'timeout', type: 'number', default: 30000 },
      { displayName: 'Body', name: 'body', type: 'json', default: '{}',
        displayOptions: { show: { method: ['POST'] } } },   // hidden unless POST
    ],
  },
  async execute(ctx) {
    const out = [];
    const items = ctx.getInputData();
    for (let i = 0; i &lt; items.length; i++) {
      const res = await fetch(ctx.getNodeParameter('url', i), {
        method: ctx.getNodeParameter('method', i),
        signal: AbortSignal.timeout(ctx.getNodeParameter('timeout', i)),
      });
      out.push({ json: await res.json() });
    }
    return [out];
  },
};
```

2 Hand every description to the editor

apps/api/server.js

```
import { nodeTypes } from '../../packages/nodes/index.js';
app.get('/rest/node-types', (req, res) =>
  res.json(Object.values(nodeTypes).map(n => n.description)));
```

3 One component turns any description into a form

apps/editor/src/ParameterPanel.tsx

```
function isVisible(p, values) {
  const show = p.displayOptions?.show;
  if (!show) return true;
  return Object.entries(show).every(([field, allowed]) => allowed.includes(values[field]));
}

export function ParameterPanel({ properties, values, onChange }) {
  return properties.filter(p => isVisible(p, values)).map(p => {
    const v = values[p.name] ?? p.default;
    const set = (val) => onChange({ ...values, [p.name]: val });
    switch (p.type) {
      case 'string':  return &lt;Field key={p.name} label={p.displayName}&gt;
                              &lt;input value={v} onChange={e => set(e.target.value)} /&gt;&lt;/Field&gt;;
      case 'number':  return &lt;Field key={p.name} label={p.displayName}&gt;
                              &lt;input type="number" value={v} onChange={e => set(+e.target.value)}
/&gt;&lt;/Field&gt;;
      case 'boolean': return &lt;Field key={p.name} label={p.displayName}&gt;
                              &lt;input type="checkbox" checked={v} onChange={e => set(e.target.checked)}
/&gt;&lt;/Field&gt;;
      case 'options': return &lt;Field key={p.name} label={p.displayName}&gt;
                              &lt;select value={v} onChange={e => set(e.target.value)}&gt;
                                {p.options.map(o => &lt;option key={o.value} value={o.value}&gt;
{o.name}&lt;/option&gt;)}
                              &lt;/select&gt;&lt;/Field&gt;;
      default: return null;
    }
  });
}
```

4 Expressions, resolved per item

packages/engine/expressions.js

```
export function resolve(value, item, results) {
  if (typeof value !== 'string' || !value.includes('{{')) return value;
  const evaluate = (expr) => {
    const $json = item.json;
    const $node = new Proxy({}, { get: (_, name) => ({ json: results[name]?.[0]?.[0]?.json ?? {} }) });
    const $now  = new Date();
    return new Function('$json', '$node', '$now', `"use strict"; return (${expr});`)($json, $node, $now);
  };
  const whole = value.match(/^\{\{(.+)\}\}$/s);          // the field is ONE expression
  if (whole) return evaluate(whole[1]);                   // keep the real type
  return value.replace(/\{\{(.+?)\}\}/gs, (_, e) => String(evaluate(e)));
}
```

Call it inside `getNodeParameter(name, itemIndex)` so node authors never see braces.

CHECK IT WORKS
Add a new node file, restart the API, refresh the editor — its whole settings panel is there, and switching Method to POST makes the Body field appear. Nobody touched the editor.

PHASE 4 · MAKE IT START BY ITSELF STAGES 7–8 · WEEK 7 · LEAD: C2

2.4 Workflows that run without anyone pressing a button

WHO BUILDS THIS PART

C2 C1 reviews

WHAT YOU NEED TO KNOW

HTTP request and response, in detail · **cron** syntax · timezones · comfort with networking that fails for boring reasons (ports, tunnels, firewalls).

What we are doing. Building the two **triggers** from section 0.1. One is a **web address** that anything on the internet can call; the other is a **clock** inside our own server. These are the two the assignment names, and together they are what turns a tool into a platform.

webhook

What it is. A web address that belongs to one workflow. Anything that can send a request can start it.

What it does in this section. The server looks the address up, then starts the workflow with the request as its first item.

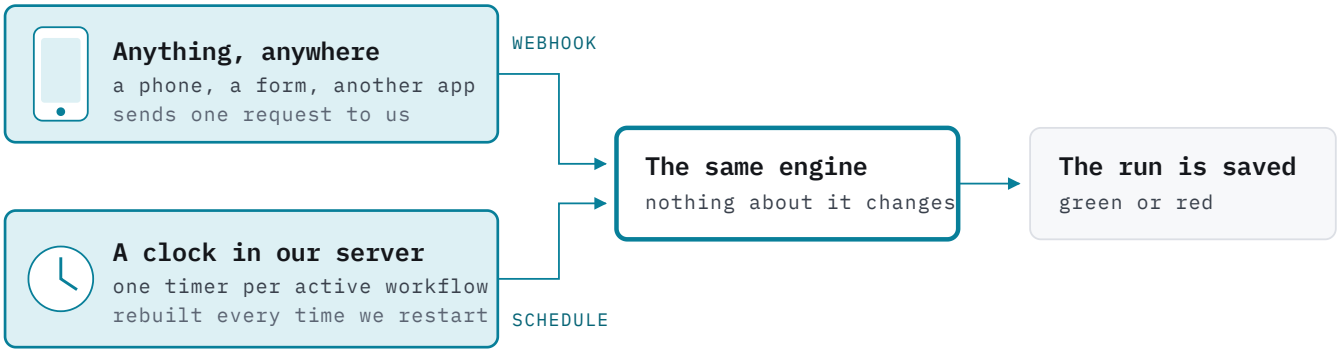
How it connects. Each workflow gets two — a test one while editing and a live one once switched on — so experiments never send real email.

cron

What it is. A five-field way of writing a repeating time. `* * * * *` means every minute.

What it does in this section. It is what a Schedule trigger stores, alongside a timezone.

How it connects. The scheduler keeps one timer per active workflow and rebuilds them all on boot — otherwise schedules stop silently after a deploy.



Each workflow gets two addresses, exactly like n8n: a test one while we are editing it, and the real one once it has been switched on.

Figure 13. Both routes arrive at the same loop. Adding a trigger never means changing the engine — that is the payoff for getting Phase 1 right.

FAMILIAR FROM N8N

The Webhook node in n8n showed you two URLs, a Test one and a Production one, and made you press "Listen for test event" before the test one would work. That is precisely the mechanism on this page, and we are copying the two-URL idea because it stops us sending real emails while we are still experimenting.

TOOLS WE USE IN THIS PHASE

`croner`	one small library that runs a function on a schedule, with timezones handled	`Postman or curl`	to fire test requests at our own addresses without a real caller
`cloudflared`	a free tunnel giving our laptop a public address while we develop		

HOW WE PROCEED

1. **Give each workflow its own web address.** When a request arrives we look up which workflow owns that address, and start it with the request handed in as the first item.
2. **Two addresses per workflow** — a test one while editing, a live one once activated.
3. **Decide what the caller gets back:** an instant "received", or wait for the workflow to finish and return its result. Both are useful, and both are easy once the first works.

4. **For schedules, keep one timer per active workflow**, and rebuild the whole set whenever the server restarts — otherwise schedules stop silently after every deploy.
5. **Always store a timezone.** Our laptops are on Australian time and the server will be on UTC, so a workflow set for 9am would otherwise fire in the evening.

DONE WHEN

A request sent from someone's phone runs a workflow on the laptop, and a scheduled workflow produces exactly one run per minute — stopping the moment it is switched off.

WATCH OUT

To test from outside while developing, use a tunnel rather than trying to open ports on the university network. It is one command and saves an afternoon.

2.4b Build it: webhooks and schedules

1 One more table, for the addresses in use

schema.sql — additions

```
CREATE TABLE webhooks (
  path        text NOT NULL,
  method      text NOT NULL,
  workflow_id uuid REFERENCES workflows(id) ON DELETE CASCADE,
  node_name   text NOT NULL,
  is_test     boolean NOT NULL DEFAULT false,
  PRIMARY KEY (path, method)
);

CREATE TABLE schedules (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  workflow_id uuid REFERENCES workflows(id) ON DELETE CASCADE,
  cron        text NOT NULL,
  timezone    text NOT NULL DEFAULT 'Australia/Sydney'
);
```

2 The webhook route

apps/api/webhooks.js

```
app.all('/webhook/:path', async (req, res) => {
  const hook = (await db.query(
    'SELECT * FROM webhooks WHERE path = $1 AND method = $2 AND is_test = false',
    [req.params.path, req.method])).rows[0];

  if (!hook) return res.status(404).json({ message: 'No workflow is listening here' });

  const trigger = [{ json: { headers: req.headers, query: req.query, body: req.body } }];

  if (hook.response_mode === 'immediately') {
    res.json({ message: 'Workflow was started' });
    runById(hook.workflow_id, trigger).catch(console.error);   // note: no await
    return;
  }
  const result = await runById(hook.workflow_id, trigger);
  res.json(lastNodeOutput(result));
});
```

The test address is the same handler with `is_test = true` and a different path prefix, so experiments never fire real emails.

3 The schedule registry

```
$ npm i croner --workspace apps/api
```

apps/api/schedules.js

```
import { Cron } from 'croner';
const jobs = new Map();

export function activate(workflow, schedule) {
  deactivate(workflow.id);
  jobs.set(workflow.id, new Cron(schedule.cron, { timezone: schedule.timezone }, () =>
    runById(workflow.id, [{ json: { timestamp: new Date().toISOString() } }])
      .catch(console.error)));
}

export function deactivate(workflowId) {
  jobs.get(workflowId)?.stop();
  jobs.delete(workflowId);
}

// rebuild every timer on boot — otherwise schedules stop silently after each deploy
export async function restoreAll(db) {
  const rows = (await db.query(
    'SELECT w.*, s.cron, s.timezone FROM workflows w JOIN schedules s ON s.workflow_id = w.id ' +
    'WHERE w.active = true')).rows;
  for (const r of rows) activate(r, r);
  console.log(`restored ${rows.length} schedules`);
}
```

4 Reach your laptop from outside

```
$ npx cloudflared tunnel --url http://localhost:5678
# prints https://something-random.trycloudflare.com — use that in Postman,
# or paste it into a phone browser to fire the webhook from another network
```

CHECK IT WORKS

A POST from your phone through the tunnel creates an execution row within a second. A workflow on `* * * * *` makes exactly one row per minute, and stops the moment you deactivate it.

PHASE 5 · MAKE IT DO THINGS STAGE 9 · WEEKS 7–8 · LEAD: C2

2.5 First, permission to use a Google account

WHO BUILDS THIS PART

C2 D2 keeps the runbook

WHAT YOU NEED TO KNOW

Reading **API documentation** carefully · the idea of **OAuth** (this page teaches it) · Node's built-in crypto · genuine care with secrets — this is the one place a mistake is expensive.

What we are doing. Getting permission to act for somebody — **without ever seeing their password**. The step most likely to cost us a week, so we do it early and once.

OAuth 2.0

What it is. The agreed way of saying "let this app act for me" without handing over a password.

What it does in this section. The seven arrows in Figure 13 are the whole of it.

How it connects. Every Google node depends on it, which is why this is the first thing built in Phase 5 and the earliest task in Week 4.

token

What it is. A temporary key a service gives you after login, used instead of a password.

What it does in this section. Google returns two: one lasting an hour, one lasting much longer.

How it connects. The long one is scrambled and stored; the shared helper in the next section quietly swaps it for a fresh short one whenever needed.

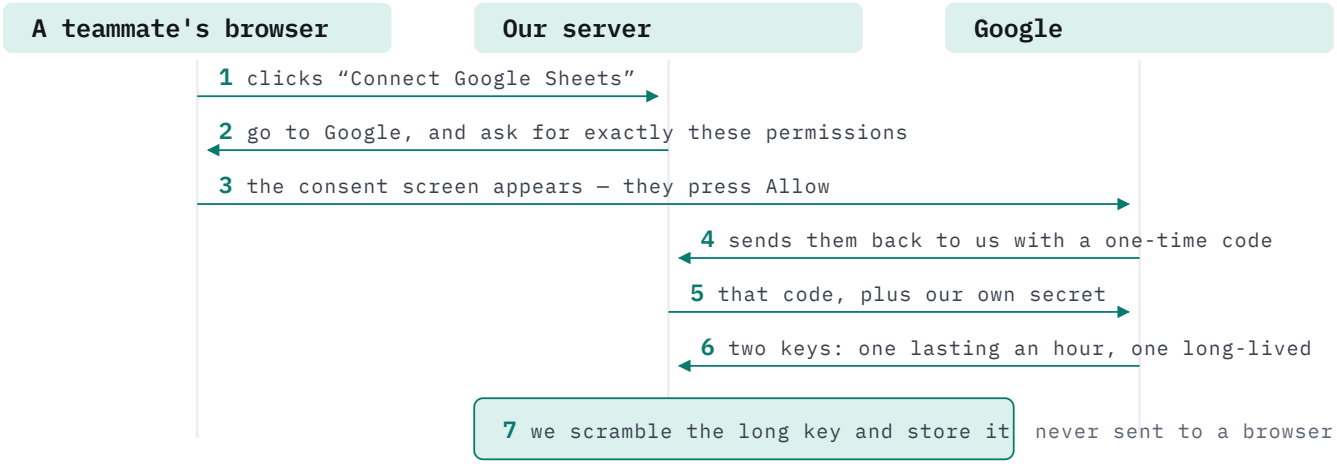


Figure 14. The whole thing in seven arrows. Steps 1–3 happen in front of the person; 4–7 happen invisibly in about half a second.

FAMILIAR FROM N8N

Adding a Google credential opened a small window, showed Google's "choose an account" screen, then said *Account connected*. These seven arrows are what happened inside it.

TOOLS WE USE IN THIS PHASE

Google Cloud Console	where the project, the four APIs and the permission screen live — free	**OAuth 2.0**	the agreed way of saying "let this app act for me" — a sequence of requests, not a library
node:crypto	built into Node; scrambles the stored keys		

HOW WE PROCEED

1. **One of us creates a Google Cloud project** and switches on the four APIs. Free, fifteen minutes.
2. **Fill in the consent screen** and keep the app in **Testing** mode. Add every group member as a **test user**, plus our demonstrator: in Testing mode nobody outside that list can connect at all.
3. **Build the round trip** from the figure above, then scramble the long-lived key before storing it — and never send it to a browser.

THE TWO TRAPS THAT END DEMOS

1 · Gmail permissions are "restricted".
Publishing the app would need a Google security review we cannot finish this semester — and we do not need one: Testing mode plus a test-user list behaves identically for us.

2 · In Testing mode the long-lived key dies after seven days.
An account connected in Week 9 is dead by Week 14, and it looks like a broken node. **Reconnecting every account is step one of the demo-morning checklist.**

DONE WHEN

Somebody presses Connect, approves on Google's screen, and our server can read a real spreadsheet with the stored keys.

2.5b Build it: the Google permission round trip

1 Create the project and switch on the four APIs

1. Go to `console.cloud.google.com` and sign in with the account that will own this
2. Click the **project dropdown** in the top bar → `NEW PROJECT` → name it `ai-automation` → `CREATE`
3. Wait for the notification, then use the same dropdown to **select the new project**. Everything after this happens inside it — this is the step people miss.
4. In the search bar type `Google Sheets API` → open it → `ENABLE`
5. Repeat step 4 for **Google Drive API**, **Google Docs API** and **Gmail API**. Four separate enables.

2 The consent screen, and the test-user list that makes it work

1. `Left menu → APIs & Services → OAuth consent screen`
2. Choose `External` → `CREATE`. Leave the publishing status as **Testing** — we never publish.
3. App name `AI Automation Platform`, your email for both support and developer contact → `SAVE AND CONTINUE`
4. `ADD OR REMOVE SCOPES` → filter and tick `.../auth/spreadsheets`, `.../auth/drive.file`, `.../auth/documents`, `.../auth/gmail.send` → `UPDATE` → `SAVE AND CONTINUE`
5. **Test users** → `ADD USERS` → paste all eight Gmail addresses **and the demonstrator's** → `SAVE AND CONTINUE`. Anybody not on this list simply cannot connect.

3 The client ID, and the exact redirect address

1. `APIs & Services → Credentials` → `CREATE CREDENTIALS` → `OAuth client ID`
2. Application type `Web application`, name it `server`
3. Under **Authorised redirect URIs** click `ADD URI` twice and paste both addresses below — character for character, including the trailing path
4. `CREATE`, then copy the **Client ID** and **Client secret** straight into `.env`. The secret is shown once.

```
http://localhost:5678/rest/oauth2-credential/callback
https://your-domain.com/rest/oauth2-credential/callback   # add the live one now too
```

4 Send the user to Google

apps/api/oauth.js

```
const SCOPES = {
  googleSheets: 'https://www.googleapis.com/auth/spreadsheets',
  googleDrive:  'https://www.googleapis.com/auth/drive.file',
  googleDocs:   'https://www.googleapis.com/auth/documents',
  gmail:        'https://www.googleapis.com/auth/gmail.send',
};

app.get('/rest/oauth2-credential/auth', (req, res) => {
  const url = 'https://accounts.google.com/o/oauth2/v2/auth?' + new URLSearchParams({
    client_id:     process.env.GOOGLE_CLIENT_ID,
    redirect_uri:  `${process.env.BASE_URL}/rest/oauth2-credential/callback`,
    response_type: 'code',
    scope:         SCOPES[req.query.type],
    access_type:   'offline',   // REQUIRED, or no long-lived key comes back
    prompt:        'consent',   // REQUIRED when re-authorising
    state:         req.query.credentialId,
  });
  res.redirect(url);
});
```

5 Trade the code for keys, and store them scrambled

apps/api/oauth.js — continued

```
app.get('/rest/oauth2-credential/callback', async (req, res) => {
  const r = await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST',
    headers: { 'content-type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({
      code:          req.query.code,
      client_id:     process.env.GOOGLE_CLIENT_ID,
      client_secret: process.env.GOOGLE_CLIENT_SECRET,
      redirect_uri:  `${process.env.BASE_URL}/rest/oauth2-credential/callback`,
      grant_type:    'authorization_code',
    }),
  });
  const tokens = await r.json();          // { access_token, refresh_token, expires_in }
  await saveCredential(req.query.state, encrypt(JSON.stringify(tokens)));
  res.send('&lt;p&gt;Connected. You can close this window.&lt;/p&gt;');
});
```

apps/api/crypto.js — no library needed

```
import crypto from 'node:crypto';
const key = Buffer.from(process.env.ENCRYPTION_KEY, 'hex');    // 32 bytes

export function encrypt(text) {
  const iv = crypto.randomBytes(12);
  const c  = crypto.createCipheriv('aes-256-gcm', key, iv);
  return { data: Buffer.concat([c.update(text, 'utf8'), c.final()]), iv, tag: c.getAuthTag() };
}
export function decrypt({ data, iv, tag }) {
  const d = crypto.createDecipheriv('aes-256-gcm', key, iv);
  d.setAuthTag(tag);
  return Buffer.concat([d.update(data), d.final()]).toString('utf8');
}
```

```
$ node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
# put the result in .env as ENCRYPTION_KEY — never commit it
```

CHECK IT WORKS

Press Connect, approve on Google's screen, and the credential row appears with unreadable bytes in it. Then call a Sheets endpoint with the decrypted access token and get a 200 back.

PHASE 5 · MAKE IT DO THINGS STAGES 10–12 · WEEKS 8–10 · D1 LEADS, SIX OF US BUILD

2.6 Then the nodes people actually came for

WHO BUILDS THIS PART

D1 leads · A1 A2 B2 C1 C2 build · one node each

WHAT YOU NEED TO KNOW

Reading **Google's API reference** · `fetch` and JSON shaping · for Gmail, patience with **MIME** · for the Code node, curiosity about how sandboxes fail.

What we are doing. Building the four Google nodes, the Code node and the AI node. With Phases 1–4 finished, each of these is a small, self-contained job — which is exactly why four of us can now work at the same time without getting in each other's way.

sandbox

What it is. A restricted place to run code you do not trust, where it cannot reach the rest of the program.

What it does in this section. Ours is a separate thread we can stop after five seconds, running the user's JavaScript in an isolated context.

How it connects. It is what makes the Code node safe enough for us — and being able to explain its limits is worth real marks in the report.

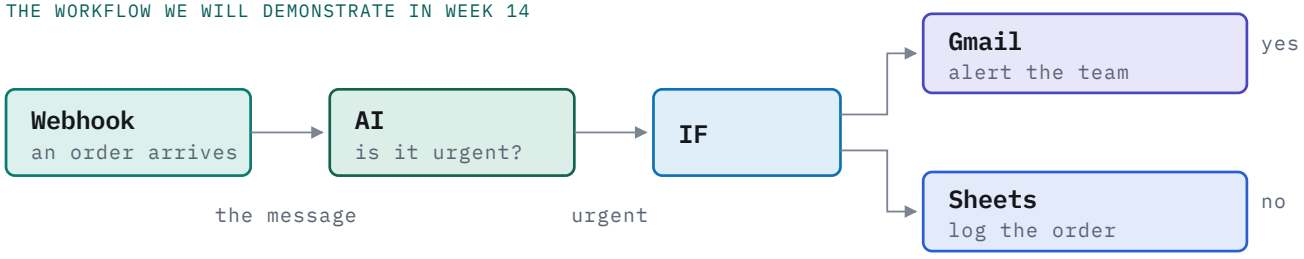
MIME

What it is. The old, strict format an email is written in before it is sent.

What it does in this section. Gmail wants the whole message as one encoded string, so we build the headers and parts by hand.

How it connects. It is the only genuinely fiddly part of Phase 5 — budget a day for it and it stops being a surprise.

THE WORKFLOW WE WILL DEMONSTRATE IN WEEK 14



Six nodes, five different phases, one screen. Everything in this document exists to make this work.

Figure 15. Each box is coloured by the phase that built it — which is also a picture of how the whole project fits together.

FAMILIAR FROM N8N

This is your weather workflow, grown up. Same shape: something starts it, something thinks about the data, something decides, and something delivers. The only new idea is the branch — and n8n's IF node worked exactly the same way.

HOW WE PROCEED

1. **Build the Google nodes easiest first: Sheets, then Docs, then Drive, then Gmail.** Each is a settings description plus one carefully built request. A single shared helper deals with expired keys, so no individual node has to think about it.
2. **Allow a full day for Gmail.** Sending mail means assembling the message in an old format by hand — knowing that in advance stops it feeling like failure.
3. **Use the Docs node for the best demo moment:** a template document with placeholders, filled in from a spreadsheet row. A mail merge in four nodes, and it looks impressive in ten seconds.
4. **The Code node's work is not running the code** — it is stopping bad code from taking the server down, which is the figure below.
5. **The AI node is one request and about half a day.** Because settings fields carry live data, somebody can type "Classify this message: `{{ … }}`" straight into the sidebar and feed the answer to an IF node. Do not leave this until last: it is what makes ours an *AI* automation platform.

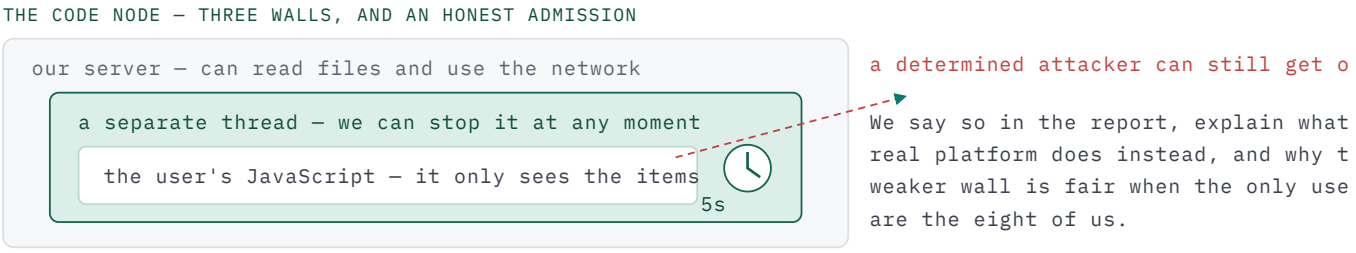


Figure 16. Being able to explain the limits of our own sandbox is worth more marks than quietly using a stronger one would have been.

DONE WHEN

The workflow above runs green from end to end, with a real email arriving and a real spreadsheet row appearing.

2.6b Build it: one helper, then four Google nodes

D1 writes the helper first. After that each node is a settings description plus one request, and six people can work in parallel without talking to each other.

1 The shared helper — refresh-on-401, written once

packages/nodes/google/request.js

```
export async function googleRequest(credentialId, url, options = {}) {
  let token = await getAccessToken(credentialId);
  const send = () => fetch(url, { ...options,
    headers: { 'content-type': 'application/json', ...options.headers,
               Authorization: `Bearer ${token}` } });

  let res = await send();
  if (res.status === 401) {                    // the hour is up — refresh once, then retry
    token = await refreshAccessToken(credentialId);
    res = await send();
  }
  if (!res.ok) throw new Error(`Google ${res.status}: ${(await res.text()).slice(0, 300)}`);
  return res.json();
}

async function refreshAccessToken(credentialId) {
  const { refresh_token } = JSON.parse(decrypt(await loadCredential(credentialId)));
  const r = await fetch('https://oauth2.googleapis.com/token', {
    method: 'POST',
    headers: { 'content-type': 'application/x-www-form-urlencoded' },
    body: new URLSearchParams({
      refresh_token, grant_type: 'refresh_token',
      client_id: process.env.GOOGLE_CLIENT_ID, client_secret: process.env.GOOGLE_CLIENT_SECRET }),
  });
  const t = await r.json();
  await cacheAccessToken(credentialId, t.access_token, t.expires_in);
  return t.access_token;
}
```

2 Google Sheets — append a row

packages/nodes/google/sheets.js

```
async execute(ctx) {
  const out = [];
  const items = ctx.getInputData();
  for (let i = 0; i &lt; items.length; i++) {
    const id    = parseSheetId(ctx.getNodeParameter('documentId', i));   // URL or bare id
    const range = ctx.getNodeParameter('sheetName', i) + '!A:Z';
    const row   = ctx.getNodeParameter('columns', i);                    // { name, email, … }

    const url = `https://sheets.googleapis.com/v4/spreadsheets/${id}/values/`
              + `${encodeURIComponent(range)}:append?valueInputOption=USER_ENTERED`;

    const body = await googleRequest(ctx.credentialId, url, {
      method: 'POST', body: JSON.stringify({ values: [Object.values(row)] }) });
    out.push({ json: body });
  }
  return [out];
}

const parseSheetId = (s) => s.match(/\/d\/([a-zA-Z0-9-_]+)/)?.[1] ?? s;
```

3 Google Docs — a mail merge in one request

packages/nodes/google/docs.js

```
const requests = Object.entries(ctx.getNodeParameter('replacements', i)).map(([k, v]) => ({
  replaceAllText: { containsText: { text: `{{${k}}}`, matchCase: true }, replaceText: String(v) },
}));

await googleRequest(ctx.credentialId,
  `https://docs.googleapis.com/v1/documents/${docId}:batchUpdate`,
  { method: 'POST', body: JSON.stringify({ requests }) });
```

Make a template document containing `{{customer_name}}` and friends, copy it with Drive first, then run this on the copy — otherwise you overwrite the template.

4 Google Drive — upload, which is multipart and slightly odd

packages/nodes/google/drive.js

```
const boundary = 'b' + crypto.randomUUID();
const meta = { name: fileName, parents: folderId ? [folderId] : undefined };

const body = Buffer.concat([
  Buffer.from(`--${boundary}\r\ncontent-type: application/json\r\n\r\n`),
  Buffer.from(JSON.stringify(meta)),
  Buffer.from(`\r\n--${boundary}\r\ncontent-type: ${mimeType}\r\n\r\n`),
  fileBuffer,
  Buffer.from(`\r\n--${boundary}--`),
]);

await googleRequest(ctx.credentialId,
  'https://www.googleapis.com/upload/drive/v3/files?uploadType=multipart',
  { method: 'POST', body, headers: { 'content-type': `multipart/related; boundary=${boundary}` } });
```

CHECK IT WORKS

Webhook → Sheets append puts a row into a spreadsheet you have open on screen, within about a second. Then Sheets read → Docs replace produces a filled document in Drive.

2.6c Build it: Gmail, the sandbox, and the AI node

5 Gmail — build the message by hand, then base64url it

This is the only genuinely fiddly node. Knowing that in advance stops it feeling like failure.

packages/nodes/google/gmail.js

```
function buildMime({ to, subject, html, attachment }) {
  const b = 'x' + crypto.randomUUID();
  const head =
    `To: ${to}\r\nSubject: ${subject}\r\nMIME-Version: 1.0\r\n` +
    `Content-Type: multipart/mixed; boundary="${b}"\r\n\r\n`;
  const bodyPart =
    `--${b}\r\nContent-Type: text/html; charset="UTF-8"\r\n\r\n${html}\r\n`;
  const filePart = attachment
    ? `--${b}\r\nContent-Type: ${attachment.mimeType}\r\n` +
      `Content-Disposition: attachment; filename="${attachment.fileName}"\r\n` +
      `Content-Transfer-Encoding: base64\r\n\r\n${attachment.data}\r\n`
    : '';
  return head + bodyPart + filePart + `--${b}--`;
}

const base64url = (s) => Buffer.from(s).toString('base64')
  .replace(/\+/g, '-').replace(/\//g, '_').replace(/=+$/, '');

await googleRequest(ctx.credentialId,
  'https://gmail.googleapis.com/gmail/v1/users/me/messages/send',
  { method: 'POST', body: JSON.stringify({ raw: base64url(buildMime(fields)) }) });
```

6 The Code node — a thread we can stop

packages/nodes/core/code.js

```
import { Worker } from 'node:worker_threads';

export function runUserCode(code, items, timeoutMs = 5000) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(`
      const vm = require('node:vm');
      const { parentPort, workerData } = require('node:worker_threads');
      const sandbox = { items: workerData.items, console: { log() {} } };
      try {
        const result = vm.runInNewContext(workerData.code, vm.createContext(sandbox),
                                           { timeout: 2000 });
        parentPort.postMessage({ ok: true, result });
      } catch (e) { parentPort.postMessage({ ok: false, error: String(e) }); }
    `, { eval: true, workerData: { code, items } });

    const timer = setTimeout(() => { worker.terminate();
      reject(new Error('Code node timed out after 5s')); }, timeoutMs);

    worker.on('message', (m) => { clearTimeout(timer);
      m.ok ? resolve(m.result) : reject(new Error(m.error)); });
    worker.on('error', (e) => { clearTimeout(timer); reject(e); });
  });
}
```

For the report: run this snippet inside the Code node and show it reaching the host — then explain what a production platform uses instead. It is the strongest security content available to us for about an hour of work.

the escape, for the report only

```
return this.constructor.constructor('return process.version')();   // vm is not a sandbox
```

7 The AI node — one request

packages/nodes/ai/anthropic.js

```
const res = await fetch('https://api.anthropic.com/v1/messages', {
  method: 'POST',
  headers: { 'x-api-key': credentials.apiKey, 'anthropic-version': '2023-06-01',
             'content-type': 'application/json' },
  body: JSON.stringify({
    model:      ctx.getNodeParameter('model', i),
    max_tokens: ctx.getNodeParameter('maxTokens', i),
    system:     ctx.getNodeParameter('systemPrompt', i),
    messages: [{ role: 'user', content: ctx.getNodeParameter('userPrompt', i) }],
  }),
});
const data = await res.json();
out.push({ json: { text: data.content[0].text } });
```

Because `userPrompt` runs through the expression resolver, somebody can type `Classify this: {{ $json.body.message }}. Answer only urgent or routine.` in the sidebar and feed the answer straight into an IF node. Point a second credential at `http://localhost:11434/v1` to use a local Ollama model instead.

> **CHECK IT WORKS**
> The Week 14 workflow runs end to end: a webhook arrives, the AI classifies it, the IF routes it, a real email lands and a real row appears.

PHASE 6 · SHIP IT STAGES 13–15 · WEEKS 11–15 · LEAD: D2 · ALL HANDS

2.7 Online, measured, and written up

WHO BUILDS THIS PART

D2 leads | all hands

WHAT YOU NEED TO KNOW

Docker Compose · the Linux command line over **SSH** · basic **DNS** (one A record) · reading a load-test summary without over-claiming.

What we are doing. Putting the platform on a real computer so our web addresses work from anywhere, then measuring it honestly against n8n, then writing the report. **At the end of Week 11 we stop adding features** — everything after that is fixing, measuring and writing.

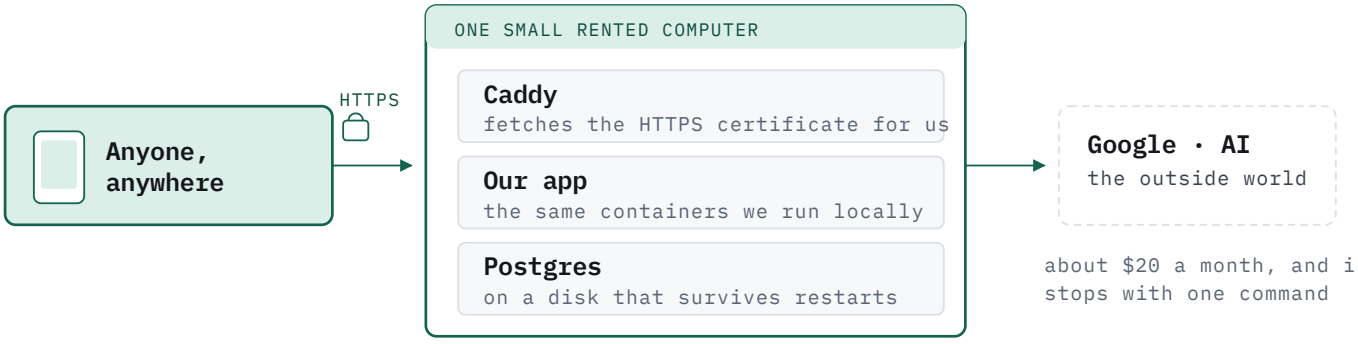


Figure 17. Not a cloud architecture diagram — one machine running the same three containers we already use. An afternoon's work, not two weeks.

FAMILIAR FROM N8N

This is simply n8n's own self-hosting story pointed at our code. The reason your `docker run n8nio/n8n` worked first time is the reason our deploy will: the container carries everything it needs with it.

TOOLS WE USE IN THIS PHASE

AWS EC2	one small rented machine that runs the whole platform	**Caddy**	gets and renews the HTTPS certificate by itself; three lines of configuration
n8n (Docker)	the platform we compare against, set up the same way as ours	**k6**	sends measured traffic at both so the comparison is real

HOW WE PROCEED

1. **Freeze features at the end of Week 11.** A group decision made in advance, so it is not an argument later.
2. **Rent one small machine, point a cheap domain at it**, and let Caddy handle HTTPS. Set a spending alarm on day one, before launching anything.
3. **Run n8n on the same machine, configured the same way** — not its simplest setup, or the comparison is unfair and a marker will notice.
4. **Build the same three workflows on both**, run each three times, and keep the raw numbers rather than only the averages.
5. **Time ourselves adding a new node to each platform.** We will win this clearly, and nobody else will think to measure it.
6. **Write honestly.** n8n does far more per run than we do, so being faster mostly means "we do less". Saying that plainly is worth more marks than the number itself.

DONE WHEN

The canvas and a webhook address both work from a phone on mobile data, the benchmark numbers are recorded, and the report draft is circulating.

WATCH OUT

Write the report from Week 5 onwards, a section at a time, and screenshot everything as it is built. You cannot re-screenshot a server you have already shut down.

2.7b Build it: online, and measured against n8n

1 The production stack — three containers and a certificate

`docker-compose.prod.yml`

```yaml
services:
  caddy:
    image: caddy:2
    ports: ["80:80", "443:443"]
    volumes: ["./Caddyfile:/etc/caddy/Caddyfile", "caddy_data:/data"]
  api:
    build: .
    env_file: .env
    environment:
      DATABASE_URL: postgres://postgres:${DB_PASSWORD}@postgres:5432/automation
    depends_on: [postgres]
  postgres:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: ${DB_PASSWORD}
      POSTGRES_DB: automation
    volumes: ["pgdata:/var/lib/postgresql/data"]
volumes:
  caddy_data: {}
  pgdata: {}
```

`Caddyfile — the entire TLS configuration`

```
your-domain.com {
  reverse_proxy api:5678
}
```

2 Rent the machine — the AWS console, click by click

1. `console.aws.amazon.com` → search `EC2` → `Instances` → `Launch instances`
2. Name `automation-prod` · image **Ubuntu Server 24.04 LTS** · instance type **t3.small**
3. **Key pair** → `Create new key pair` → RSA, `.pem` → it downloads once. Lose it and you lose the machine.
4. **Network settings** → `Edit` → allow `SSH` from **My IP**, and `HTTP` + `HTTPS` from anywhere
5. Storage **20 GiB gp3** → `Launch instance`
6. `EC2 → Elastic IPs → Allocate Elastic IP address` → `Associate` it with the instance, so the address survives a reboot
7. At your domain registrar add one **A record** pointing `your-domain.com` at that address
8. **Before anything else:** `Billing → Budgets → Create budget` → cost budget, **$50**, alert to all eight emails

```
$ chmod 400 automation-prod.pem
$ ssh -i automation-prod.pem ubuntu@your-domain.com
$ curl -fsSL https://get.docker.com | sh && sudo usermod -aG docker ubuntu
$ exit && ssh -i automation-prod.pem ubuntu@your-domain.com   # re-login for the group
```

3 Put the project on it

```
$ ssh ubuntu@your-server
$ git clone https://github.com/<org>/ai-automation && cd ai-automation
$ cp .env.example .env && nano .env        # real keys go here only
$ docker compose -f docker-compose.prod.yml up -d --build
# later deploys:
$ git pull && docker compose -f docker-compose.prod.yml up -d --build
```

4 Stand n8n up beside it, configured the same way

benchmarks/n8n/docker-compose.yml

```yaml
services:
  n8n:
    image: n8nio/n8n:1.70.1          # pin it, and cite this version in the report
    environment:
      DB_TYPE: postgresdb
      DB_POSTGRESDB_HOST: n8n-db
      EXECUTIONS_MODE: queue
      QUEUE_BULL_REDIS_HOST: redis
    ports: ["5679:5678"]
    deploy: { resources: { limits: { cpus: "1.0", memory: 1G } } }
  n8n-worker:
    image: n8nio/n8n:1.70.1
    command: worker
    deploy: { replicas: 2, resources: { limits: { cpus: "1.0", memory: 1G } } }
```

Give our own containers the identical `cpus` and `memory` limits, or the comparison is not a comparison.

5 Measure both with the same script

benchmarks/k6/w1.js

```
import http from 'k6/http';
export const options = {
  scenarios: { load: { executor: 'constant-vus', vus: 10, duration: '60s', gracefulStop: '10s' } },
  thresholds: { http_req_duration: ['p(95)&lt;1000'] },
};
export default function () {
  http.post(`${__ENV.TARGET}/webhook/bench-w1`, JSON.stringify({ id: __VU, ts: Date.now() }),
            { headers: { 'content-type': 'application/json' } });
}
```

```
$ k6 run -e TARGET=http://localhost:5678 --summary-export=ours-w1.json benchmarks/k6/w1.js
$ k6 run -e TARGET=http://localhost:5679 --summary-export=n8n-w1.json  benchmarks/k6/w1.js
# three runs each, commit every JSON file — the raw data is what makes it credible
```

CHECK IT WORKS

The editor and a webhook address both work from a phone on mobile data, and you have six JSON result files for workflow 1. Then repeat for workflows 2 and 3.

3.1 How six phases become one working system

WHO BUILDS THIS PART

D2 one of the eight rotates each week

WHAT YOU NEED TO KNOW

git branches and merges · npm workspaces · Docker Compose · the discipline to run the whole thing from a clean clone every Thursday.

Each phase produced a piece, and each of us has been working in our own folder. Integration is not a week at the end — it is a habit from Week 4: **one repository, one command, and everything runs.** If assembling the project is ever difficult, something went wrong earlier.

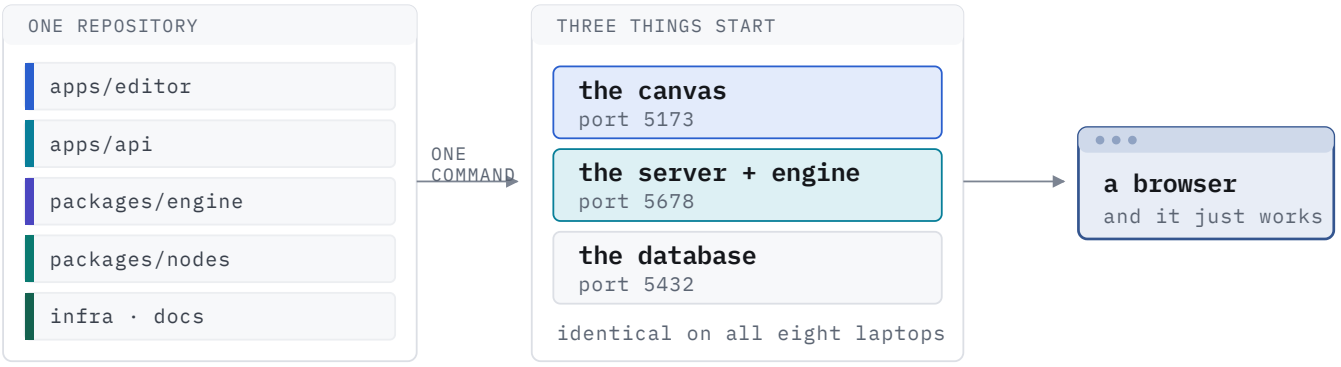


Figure 18. Five folders, one command, three programs. Anyone who can clone the repository can run the whole platform.

HOW WE PROCEED

1. **One repository from day one**, everyone on their own branch, merging into the main one. Never a zip file passed around, and never "I will send you my folder".
2. **One command starts everything.** If that command ever stops working, fixing it matters more than any feature.
3. **One example settings file** in the repository listing every key the project needs, with fake values. Real keys live only in each person's own untracked copy.
4. **A starter workflow loaded on first run**, so a fresh clone shows something working instead of an empty screen.
5. **Integration every Thursday.** Everything merges, and one person — a different one of the eight each week — clones the repository fresh and runs it. Whatever breaks is that week's priority, and D2 chairs it.

DONE WHEN

Someone who has never touched the project can clone it, run one command, and have a working platform in under ten minutes without asking a question.

WATCH OUT

The classic failure is eight people whose parts each work alone and have never run together — and with eight it happens faster. Merging weekly from Week 4 makes integration boring, which is exactly what we want it to be.

3.1b Build it: one repository, one command

1 The layout everyone clones

the whole tree

```
ai-automation/
├── apps/
│   ├── api/          # Express, the five routes, webhooks, schedules, OAuth   C1 C2
│   └── editor/       # React + React Flow + the parameter panel               B1 B2
├── packages/
│   ├── engine/       # the loop, items, expressions                           A1
│   └── nodes/
│       ├── core/     # IF, Set, Merge, Code                                   A2
│       ├── google/   # Sheets, Drive, Docs, Gmail                            D1
│       └── ai/       # the LLM node                                           A1
├── benchmarks/       # k6 scripts, the n8n stack, raw results                 D2
├── infra/            # Caddyfile, docker-compose.prod.yml, deploy notes       D2
├── docs/             # this guide, ADRs, the report as it is written          D2
├── docker-compose.yml
└── .env.example
```

2 One command starts everything

package.json — root

```
{
  "type": "module",
  "workspaces": ["apps/*", "packages/*"],
  "scripts": {
    "dev":     "docker compose up -d && npm run dev --workspaces --if-present",
    "migrate": "psql $DATABASE_URL -f schema.sql && psql $DATABASE_URL -f seed.sql",
    "test":    "vitest run",
    "e2e":     "playwright test",
    "lint":    "eslint . && tsc --noEmit"
  }
}
```

```
$ git clone … && cd ai-automation
$ cp .env.example .env
$ npm install
$ npm run migrate
$ npm run dev          # editor 5173 · api 5678 · postgres 5432
```

Five commands, ten minutes, no questions asked of anybody. That is the target.

3 Every key the project needs, with fake values

.env.example — committed; .env is never committed

```
DATABASE_URL=postgres://postgres:dev@localhost:5432/automation
BASE_URL=http://localhost:5678
ENCRYPTION_KEY=0000000000000000000000000000000000000000000000000000000000000000
GOOGLE_CLIENT_ID=replace-me.apps.googleusercontent.com
GOOGLE_CLIENT_SECRET=replace-me
ANTHROPIC_API_KEY=replace-me
```

```
$ echo ".env" >> .gitignore
$ npx gitleaks detect --no-git   # run it once a week; a leaked key costs marks
```

4 A starter workflow, so a fresh clone shows something

seed.sql

```
INSERT INTO workflows (name, nodes, connections) VALUES (
  'Hello weather',
  '[{"name":"Start","type":"manualTrigger","parameters":{},"position":{"x":80,"y":80}},
    {"name":"Weather","type":"httpRequest",
     "parameters":{"url":"https://wttr.in/Newcastle?format=j1"},"position":{"x":360,"y":80}}]',
  '{"Start":[["Weather"]]}'
);
```

The difference between a new teammate being productive in ten minutes or in two days.

CHECK IT WORKS
Every Thursday, one of the eight — a different one each week — clones the repository into a brand-new folder and runs those five commands. Whatever breaks is that week's first priority.

3.2 Four levels of testing, and what each one catches

WHO BUILDS THIS PART

D2 sets it up | every owner writes their own

WHAT YOU NEED TO KNOW

Vitest · a little Node `http` for the fake server · **Playwright** · **YAML** for the CI file.

Not testing everything — testing the things that break quietly. The rule: **the lower the level, the more tests and the faster they run.** We do not need all four from day one; each arrives with the phase that needs it.

1 · Engine tests about 20 · milliseconds	Does the loop run nodes in the right order? Does a branch with zero items correctly stop everything after it?
2 · Node tests one per node · seconds	Does the Gmail node build the right request from its settings? Runs against a fake server, never real Google.
3 · Browser test one or two · a minute	Can a person drag out two nodes, connect them, press Run and see the result? One test covering the whole path.
4 · The real run by hand · before demos	Does a real email actually arrive? Does a row appear in a real spreadsheet? Only a person can confirm this.

Levels 1–3 run automatically on every change. Level 4 is a checklist we work through before every demo.

Figure 19. Each level catches a kind of bug the level above it cannot. Skipping level 2 is why integrations break silently the week before a demo.

TOOLS WE USE FOR TESTING

Vitest	runs levels 1 and 2 — fast, and the default for a Node project	**A fake server**	a tiny local program that answers like Google does, so node tests never touch the real thing
Playwright	drives a real browser for level 3	**GitHub Actions**	runs levels 1–3 on every change, so nobody has to remember

HOW WE PROCEED

1. **Start with the engine (level 1), in Phase 1.** The cheapest tests we will ever write, and they protect the piece everything else depends on.
2. **Add one test per node as we build it (level 2)**, not afterwards. The trick is the fake server: our node calls a local address instead of Google's, so the test is fast, free, and works with no internet.
3. **Write exactly one browser test (level 3)** once the canvas can save. One is enough — browser tests are slow and break for boring reasons.
4. **Keep level 4 as a written checklist**, not as tests. Section 3.3 is that checklist.
5. **Turn the automatic checks on early**, so that from Week 5 a broken change is caught when it is pushed rather than the night before the demo.

DONE WHEN

Someone pushes a change that breaks the IF node and a red cross appears within two minutes — without anybody noticing by hand.

DO NOT TEST

Google's APIs, React Flow, or Postgres. They are not our code and testing them wastes time we do not have. Test *our* logic and use a fake for everything else.

3.2b Build it: the four levels, in code

```
$ npm i -D vitest @playwright/test
$ npx playwright install chromium
```

1 Level 1 — the engine, in milliseconds

packages/engine/engine.test.js

```
import { test, expect } from 'vitest';
import { runWorkflow } from './engine.js';

const fake = (name, out) => ({ description: { name, group: 'action', properties: [] },
                               async execute() { return out; } });

test('runs nodes in order and passes items along', async () => {
  const types = { start: fake('start', [[{ json: { a: 1 } }]]),
                  next:  fake('next',  [[{ json: { b: 2 } }]]) };
  types.start.description.group = 'trigger';
  const wf = { nodes: [{ name: 'S', type: 'start', parameters: {} },
                       { name: 'N', type: 'next',  parameters: {} }],
               connections: { S: [['N']] } };
  const r = await runWorkflow(wf, types);
  expect(r.N[0][0].json).toEqual({ b: 2 });
});

test('an empty branch stops everything after it', async () => {
  const types = { start: fake('start', [[]]), never: fake('never', [[{ json: {} }]]) };
  types.start.description.group = 'trigger';
  const wf = { nodes: [{ name: 'S', type: 'start', parameters: {} },
                       { name: 'X', type: 'never', parameters: {} }],
               connections: { S: [['X']] } };
  const r = await runWorkflow(wf, types);
  expect(r.X).toBeUndefined();          // it never ran — and that is correct
});
```

2 Level 2 — a node against a fake Google, with no internet

packages/nodes/google/sheets.test.js

```
import { test, expect, beforeAll, afterAll } from 'vitest';
import http from 'node:http';

let server, received;
beforeAll(() => new Promise(done => {
  server = http.createServer((req, res) => {
    let body = ''; req.on('data', c => body += c);
    req.on('end', () => {
      received = { url: req.url, method: req.method, body: JSON.parse(body || '{}') };
      res.writeHead(200, { 'content-type': 'application/json' });
      res.end(JSON.stringify({ updates: { updatedRows: 1 } }));
    });
  }).listen(4010, done);
}));
afterAll(() => server.close());

test('append sends one row, in order, to the right range', async () => {
  await sheetsNode.execute(makeCtx({
    base: 'http://localhost:4010',
    documentId: 'https://docs.google.com/spreadsheets/d/ABC123/edit',
    sheetName: 'Orders', columns: { name: 'Alice', email: 'a@example.com' },
  }));
  expect(received.method).toBe('POST');
  expect(received.url).toContain('/ABC123/values/');
  expect(received.body.values).toEqual([['Alice', 'a@example.com']]);
});
```

Thirty lines, and every Google node gets the same treatment. No credentials, no quota, no network — so it runs on every push in about two seconds.

CHECK LEVELS 1 AND 2

`npm test` finishes in under five seconds, with no internet connection and no Google credentials. If it needs either, the fake server is not being used.

3.2c Build it: the browser test, and running it all automatically

3 Level 3 — one browser test, and only one

e2e/build-and-run.spec.ts

```
import { test, expect } from '@playwright/test';

test('draw two nodes, connect them, run, see a result', async ({ page }) => {
  await page.goto('http://localhost:5173');
  await page.getByRole('button', { name: 'Add node' }).click();
  await page.getByText('Manual Trigger').click();
  await page.getByRole('button', { name: 'Add node' }).click();
  await page.getByText('Edit Fields').click();
  await page.getByTestId('handle-Manual Trigger-0').dragTo(page.getByTestId('target-Edit Fields'));
  await page.getByRole('button', { name: 'Save' }).click();
  await page.getByRole('button', { name: 'Run' }).click();
  await expect(page.getByTestId('output-Edit Fields')).toContainText('json');
});
```

4 Run levels 1–3 on every push

.github/workflows/ci.yml

```
name: ci
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: dev, POSTGRES_DB: automation }
        ports: ["5432:5432"]
        options: --health-cmd pg_isready --health-interval 5s --health-retries 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 22, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npx playwright install --with-deps chromium
      - run: npm run e2e
```

CHECK IT WORKS

Break the IF node on purpose, push, and a red cross appears within two minutes — without anybody noticing by hand. That is the whole point of levels 1–3.

3.3 Proving the whole thing works, on the day

WHO BUILDS THIS PART

D2 directs · A1 · D1 present

WHAT YOU NEED TO KNOW

Nothing technical — a printed checklist, a second laptop, a phone hotspot, and somebody willing to say "stop, do it again".

The last two weeks are not for building. They are for making sure that what we built works **in front of somebody else, on a day we do not control.**

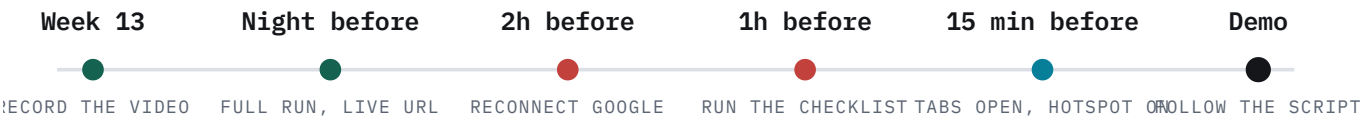


Figure 20. The two red markers are the ones that have ended other groups' demos. Neither takes more than fifteen minutes.

THE ACCEPTANCE CHECKLIST — EVERYTHING THE BRIEF ASKS FOR, AND HOW WE PROVE IT

WHAT IS REQUIRED	WHAT WE SHOW
Schedule trigger	A workflow set to every minute; the run list fills up while we talk, and stops when we switch it off.
Webhook	Somebody in the room sends a request from their phone; the run appears on screen immediately.
HTTP request	The demo workflow calls a real outside API and shows what came back.
Google Sheets	A row appears in a spreadsheet that is open on screen.
Google Docs	A template document comes back with its placeholders filled in from that row.
Google Drive	That finished document appears in a Drive folder.
Gmail	A real email arrives in somebody's inbox, with the document attached.
Code node	A snippet turns one item into ten; then an endless loop is stopped after five seconds and the server stays up.
AI node	The model classifies an incoming message and the IF node routes it to the right branch.
Extensibility	A brand-new node is written live in about ninety seconds and appears with a full settings panel.
Comparison with n8n	One slide: our numbers beside n8n's, the raw data available, and an honest sentence about why we are faster.

DONE WHEN

We have run this entire checklist twice, on the deployed address, on two different machines — and the fallback video exists.

THE LAST THING TO SAY

Every row above maps to a sentence in the brief. If we can tick all eleven, the demo is finished — everything else is polish, and polish is not what is being marked.

4.1 Eight people, eight areas

We own **parts of the system**, not tasks — so nobody has to ask whose decision something is, and we rarely edit the same file at the same time. Eight people means **four areas of two**: within a pair you swap work freely and review each other; across pairs you talk through the written agreements.

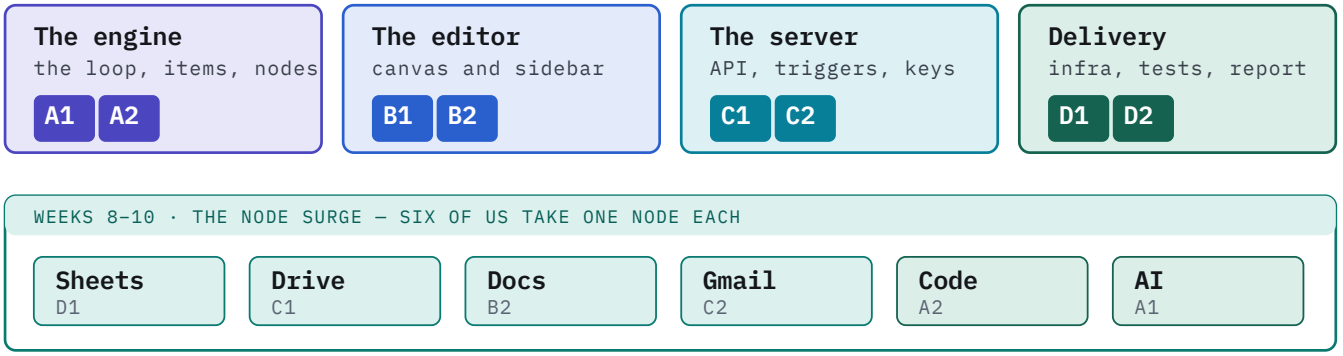


Figure 21. Areas are owned all semester. The nodes in Weeks 8–10 are a *surge* — six of us take one each, which is only possible because Phase 3 made nodes independent of the editor.

	OWNS	RESPONSIBLE FOR, ALL SEMESTER
A1	`packages/engine`	**Engine core and technical lead.** The loop, the item model, the expression resolver. Writes Phase 1 alone. Holds the casting vote on architecture. Takes the AI node in the surge.
A2	`packages/nodes/core`	**Control flow and isolation.** IF, Set, Merge, per-node retry and error handling; later the Code node's sandbox. Reviews A1's work and vice versa.
B1	`apps/editor/canvas`	**The canvas.** React Flow, node rendering, and the conversion between its format and ours — the fiddliest code in the editor. Later the code-editing box.
B2	`apps/editor/panel`	**The sidebar and run viewer.** The settings-panel generator — the single highest-leverage piece in the project — plus the execution list and log. Takes Docs in the surge.
C1	`apps/api`	**Server and database.** The five addresses, the schema and its migrations, and the query layer. Takes Drive in the surge.
C2	`apps/api/triggers`	**Triggers and credentials.** Webhook ingress, the schedule registry, encryption, and the Google permission flow. Takes Gmail in the surge — they already know that auth path.
D1	`packages/nodes/google`	**Integrations lead.** The shared Google request helper and the node-writing conventions everyone else copies. Takes Sheets first, so the pattern exists before the surge starts.
D2	`infra/ · docs/`	**Infrastructure, testing and the report.** Docker, automatic checks, deployment, the benchmark. Editor-in-chief of the report and director of the demo. Starts on day one, not in Week 12.

> **THREE WRITTEN AGREEMENTS HOLD EIGHT PEOPLE TOGETHER**
> The **node shape** (frozen Phase 1, A1 owns it), the **five addresses** (frozen Phase 2, C1 owns them), and the **settings-description format** (frozen Phase 3, B2 owns it). Everything else can change without a meeting. These three cannot change without one.

4.2 The hard part of being eight, and how we solve it

Eight people is an advantage in Weeks 8 to 10 and a **liability in Weeks 4 to 6**. Phase 1 is genuinely one person's work and Phase 2 is about three. If we do not plan for that, five of us spend three weeks either idle or — much worse — rewriting each other's code.

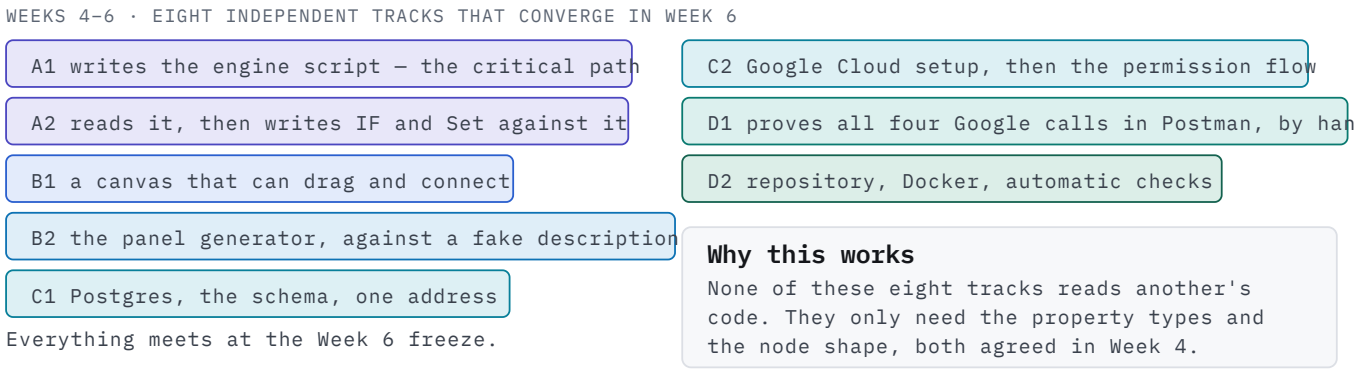


Figure 22. Eight tracks, no shared code. The only Week 4 decisions everyone needs are the node shape and the list of setting types — both are one meeting.

THE TRICK FOR B2

Build against a fake

The settings generator does not need real nodes — it needs a *description*. Write three fake ones by hand in Week 4 and build the whole panel against them. When real nodes arrive in Week 6 they simply work.

THE TRICK FOR D1

Prove the API calls by hand first

Every Google call can be made in Postman before a single node exists. Doing all four in Week 4 — and saving the exact requests — turns Weeks 8–10 from research into typing, and finds the permission problems while there is still time.

HOW EIGHT PEOPLE WORK WITHOUT GETTING IN EACH OTHER'S WAY

1. **Directory ownership is real.** A change inside someone's directory is reviewed by them. One approval plus green checks merges it; nobody waits for all eight.
2. **Pairs review each other.** A1↔A2, B1↔B2, C1↔C2, D1↔D2. Reviews stay fast because the reviewer already knows the code.
3. **Two short meetings, not one long one.** Monday, twenty minutes, in two groups — engine and editor, then server and delivery. Thursday, forty minutes, all eight, and everyone shows something running.
4. **Never eight people on one problem.** If a thing is stuck, at most two people work on it and the rest keep moving. Eight people debugging one webhook is a wasted afternoon for six.
5. **Write the contribution statement weekly**, from the commit history, while it is still accurate. With eight names it is a marked component and it is impossible to reconstruct in Week 15.

> **WHAT EIGHT BUYS US**
> Capacity in Weeks 8–10, and therefore scope. With five people the AI Agent node, the Merge node, per-node retry and the execution queue were all "only if we are ahead". With eight they are **expected** — and they are the four things most likely to make our comparison with n8n interesting rather than apologetic.

4.3 Three risks that end projects, and six habits that save them

Google keys expire after seven days

While our app is in Testing mode the long-lived key stops working after a week. Connected in Week 9, dead by Week 14 — and it looks like a broken node, not an expired key.

C2 owns this: reconnect every Google account on the morning of the demo. Step one of the checklist, not a footnote.

Gmail permissions are restricted

Making the app public needs a Google security review we cannot complete this semester.

C2 owns this: stay in Testing mode and add all eight of us — plus our demonstrator — as test users in Week 4, not Week 8.

Eight people, five of them idle

The failure specific to a large group: too few tracks early, so people either wait or start rewriting the core.

A1 owns this: the eight tracks on the previous page are assigned in Week 4, before anyone asks what to do.

Six habits that will save us weeks

01 **Build the boring version first.** An HTTP node that only does the simplest request is worth more in Week 6 than a perfect one in Week 9, because seven other people can then start.

02 **One person owns the engine.** Two people editing it at once produces a mess nobody can untangle — which is exactly why A2 reviews it rather than writing it.

03 **We are not building user accounts.** Logins and password resets earn no marks here and will eat a week of somebody's time.

04 **When a node will not work, test the plain request in Postman first.** Most "my node is broken" turns out to be a missing permission or a wrong header — nothing to do with our code.

05 **Print the data between nodes, constantly.** Almost every confusing bug is the shape of the data, not the logic — and section 1.4 explains the shape it should have.

06 **Copy n8n's good ideas deliberately, and say so in the report.** Explaining *why* a design is right is exactly what the assignment asks for. Copying without understanding is what costs marks.

> **IF WE FALL BEHIND**
> We cut *nodes*, never phases. Losing the Drive node costs one row in the acceptance checklist. Losing Phase 3 means every later node needs a hand-built settings panel and the six-person surge becomes a queue behind one person. The order of what to drop, if it comes to it: the Merge node, then Drive, then Docs, then the second half of the benchmark. Never the AI node, and never a phase.

4.4 What each of us does this week

Eight tasks, one per person, none of which needs anybody else's code to exist first. Phase 1 cannot properly start until the first two are done.

	TASK	BLOCKS
A1	**Write the engine script** from Figure 5 and walk all eight of us through it. Then write the node shape down and freeze it.	everything
A2	Read A1's script line by line, then write the IF and Set nodes against the agreed shape. Nothing that calls an outside service.	Phase 3
B1	A React Flow canvas that can add a box and connect two boxes. Nothing needs to save yet.	Phase 2
B2	Agree the list of setting types with A1, then build the panel generator against three **hand-written fake descriptions**.	Phase 3
C1	Postgres in a container, the two tables, and a server that answers one address.	Phase 2
C2	**Create the Google Cloud project**, switch on the four APIs, fill in the consent screen, add all eight of us and the demonstrator as test users.	Phase 5
D1	**Prove all four Google calls by hand in Postman** — append a row, fill a template, upload a file, send a message — and save the exact requests in the repository.	Phase 5
D2	Repository set up, the shared Docker file, automatic checks on every change, an AWS account with a spending alarm, and the report skeleton.	Phases 2 and 6
All	Read Parts 0 and 1 before Monday, and put the Week 13 rehearsal in the calendar now.	—

WHY C2'S AND D1'S TASKS ARE ON THIS LIST

Both look like Week 8 work. They are here because Google's permission screen has waiting periods we do not control, restricted permissions we cannot argue with, and a test-user list that must include our demonstrator — and because proving the four API calls by hand now turns three weeks of research into three weeks of typing.

OUR WEEKLY RHYTHM FROM HERE

Monday, twenty minutes, in two groups: engine and editor, then server and delivery. Thursday, forty minutes, all eight — everyone shows what actually runs, and one person clones the repository fresh and starts it. Nothing else.

THE ONE SENTENCE TO REMEMBER

Every phase ends in something we can run and show. If a phase takes twice as long as this document says, that is normal — we cut a node, never a phase, and we never start a new phase before the previous one's "done when" is actually true.